



Faculty of Computer Science and Information Technology

Personalised Chatbox for Role-Based Guidance and Group Accountability

Lleyton Geboh Anak Leslie
(84354)

Bachelor of Computer Science with Honours (Multimedia Computing)

2026

ACKNOWLEDGEMENT

I would like to express gratitude to my Final Year Project supervisor, Ts. Nurfaiza bt Jali for the continuous guidance, constructive feedback, and support throughout the development of this Final Year Project. Her insights, supervision and suggestions were very useful in shaping the direction and the scope of the study.

I would also like to express my appreciation to my examiner, Dr. Nuha binti Loling Othman, and the Final Year Project coordinator for their valuable comments, guidance, and support throughout the completion of this project.

Appreciation is also extended to the respondents who helped participate in the questionnaire survey and provided feedback during the requirement analysis phase. Their cooperation contributed significantly to the understanding of the issues which are related to the group assignments and the design of the proposed system.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	v
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
ABSTRACT	ix
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Scope	2
1.4 Aims and Objectives	2
1.5 Brief Methodology	2
1.6 Significance of the Project	3
1.7 Expected Outcome	4
1.8 Summary	4
CHAPTER 2: LITERATURE REVIEW	5
2.1 Overview	5
2.2 Overview of Objective	5
2.3 Background	6
2.4 System Continuation from Previous Work	7
2.5 Existing Applications	9
2.5.1 Trello	9
2.5.2 Google Classroom	10
2.5.3 Jira	11
2.5.4 Existing Application or System Comparison	12
2.6 Review on Tools and Technology	13
2.6.1 Software	13
2.6.1.1 TypeScript	13
2.6.1.2 ReactJS	13
2.6.1.3 NodeJS and ExpressJS	13
2.6.1.4 MongoDB	13
2.6.1.5 Redis	14
2.6.1.6 Docker (Docker Desktop)	14
2.6.2 Hardware	14

2.7 Summary	14
CHAPTER 3: METHODOLOGY	15
3.1 Overview	15
3.2 Rapid Prototyping Model	15
3.2.1 Planning	16
3.2.2 Analysis	16
3.2.2.1 Analysis of the Current Method	17
3.2.2.2 Analysis on Proposed Application	18
3.2.2.3 Questionnaires	19
3.2.2.4 Software Requirements	29
3.2.2.5 Hardware Requirements	30
3.2.2.6 Design and Prototyping (UML)	31
3.2.2.7 Wireframes	32
3.2.3 Prototype Cycle	35
3.2.3.1 Develop	36
3.2.3.2 Demonstrate	36
3.2.3.3 Refine	36
3.2.4 Testing	37
3.2.5 Implementing	37
3.3 Summary	37
CHAPTER 4: IMPLEMENTATION	39
4.1 Overview	39
4.2 Installation and Configuration of System's Components	39
4.2.1 React	39
4.2.2 Node.js and Express	42
4.2.3 MongoDB	46
4.2.4 ChatGPT API	48
4.2.5 Docker and Nginx	50
4.3 Role Based Access	51
4.4 Workflow for System's Application	53
4.5 Modules for System's Application	56
4.5.1 Chatbox Interaction Module	56
4.5.2 Task Management Module	56
4.5.3 AI Response Module	57
4.5.4 Reminder and Notification Module	58
4.5.5 Database Management Module	59
4.6 Summary	60

CHAPTER 5: TESTING	61
5.1 Overview	61
5.2 Functional Testing	61
5.2.1 System Testing	61
5.2.2 Unit Testing	62
5.3 Non-Functional Testing	63
5.3.1 Usability testing	63
5.3.1.1 System Functionality of the Proposed System	63
5.3.1.2 User Interface Design of the Proposed System	63
5.4 Summary	64
CHAPTER 6: CONCLUSION AND FUTURE WORKS	65
6.1 Overview	65
6.2 Objective Achievement	65
6.3 Project Limitations	65
6.4 Conclusion	66
6.5 Future Works	67
REFERENCES	68
APPENDIX A	70

LIST OF FIGURES

Figure 1.1: Rapid Prototyping Model (Alaoui et al., 2020)	3
Figure 2.1: Use Case Diagram of the SeLab Management System	8
Figure 2.2: Existing System - Trello	10
Figure 2.3: Existing System – Google Classroom	11
Figure 2.4: Existing System - Jira	12
Figure 3.1: Age Distribution of Respondents	20
Figure 3.2 : Gender Distribution of Respondents	20
Figure 3.3: Academic Status of Respondents	21
Figure 3.4: Experience with AI Chatbots	21
Figure 3.5: Frequency of Chatbot Usage	22
Figure 3.6: Purpose of Chatbot Usage	22
Figure 3.7: Frequency of Group Project Participation	23
Figure 3.8: Issues in Group Communication and Accountability	23
Figure 3.9: Task Completion Timeliness in Group Projects	24
Figure 3.10: Frequency of Task and Deadline Reminders	24
Figure 3.11: Effectiveness of Responsibility Tracking	25
Figure 3.12: Perceived Improvement in Group Communication	25
Figure 3.13: Perception of Role-Based Guidance	26
Figure 3.14: Usefulness of Task and Deadline Reminders	26
Figure 3.15: Reduction of Forgotten Responsibilities	27
Figure 3.16: Perceived Increase in Accountability	27
Figure 3.17: Organisation of Group Work	28
Figure 3.18: Accountable Group Work Environment	28
Figure 3.19: Fairness and Transparency of Responsibility Distribution	29
Figure 3.20: Use Case Diagram	31
Figure 3.21: Wireframe of the Personalised Chatbox Interface (Leader Initial View)	33
Figure 3.22: Wireframe of the Personalised Chatbox Interface (Member Initial View)	33
Figure 3.23: Wireframe of Leader Task Management via Personalised Chatbox	34
Figure 3.24: Wireframe of Member Task and Reminder View via Personalised Chatbox	35
Figure 4.1: Installation of project dependencies using npm in the terminal	40
Figure 4.2: React application running successfully with a local development URL	40
Figure 4.3: Chatbox interface displayed in the system	41

Figure 4.4: Styled chat interface with formatted messages	42
Figure 4.5: Installation of backend dependencies in the terminal	43
Figure 4.6: Backend server running successfully	43
Figure 4.7: Backend API endpoint receiving chat request from frontend	44
Figure 4.8: Backend processing request and returning response using ChatGPT API	44
Figure 4.9: Communication flow between the frontend, backend, database and ChatGPT API	45
Figure 4.10: Successful connection between the backend server and MongoDB	47
Figure 4.11: Collections created in MongoDB for the proposed system	47
Figure 4.12: Example of a response generated by the ChatGPT API in the chatbox interface	49
Figure 4.13: Docker containers running successfully	50
Figure 4.14: Leader interface in the chatbox system	52
Figure 4.15: Member interface in the chatbox system	52
Figure 4.16: Overall workflow of the personalised chatbox system	55
Figure 4.17: Chatbox Interaction Interface	56
Figure 4.18: Task Management Functionality	57
Figure 4.19: AI-Generated Response in Chatbox	58
Figure 4.20: Reminder and Notification Display	59
Figure 4.21: Database Structure for the Chatbox System	60

LIST OF TABLES

Table 2.1: Comparison of Existing System	12
Table 3.1: Software requirements for the proposed personalised chatbox system	30
Table 5.1: System Testing Results	62
Table 5.2: Unit Testing Results	62
Table 6.1: Objective Achievement	65

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
MongoDB	Mongo Database
NodeJS	Node JavaScript
npm	Node Package Manager
ReactJS	React JavaScript
Redis	Remote Dictionary Server
UML	Unified Modeling Language
UI	User Interface
UNIMAS	Universiti Malaysia Sarawak
URL	Uniform Resource Locator
UniMAP	Universiti Malaysia Perlis

ABSTRACT

Group assignments are an essential part of higher education, but students frequently face issues including unclear task distribution, poor communication, missed deadlines, and free-riding actions. While providing little support for role-based guidance and group accountability, the current educational systems mostly focus on content delivery and assignment submission. The primary goal of the project is to produce, develop and evaluate the personalised chatbox system which provides task accountability and role-based guidance in an educational environment. The project has two goals: first, which is to analyse the current methods and the systems that are employed by students for managing group work, and to produce and develop a role-based personalised chatbox which facilitates communication, task reminders and progress monitoring; second, to test and evaluate the usability and the efficiency of the proposed system in enhancing teamwork and accountability within a group. The Rapid Prototyping Model is used in the system's development to enable iterative refinement that depends on user input. Prior creation, a questionnaire survey was sent to people in order to collect the requirements and insights into group work issues and chatbox usage. Based on the findings, a prototype of the personalised chatbox system was produced and incorporated into an educational website. The proposed system's intention is to improve group coordination, improve group accountability, and decrease free-riding behaviour through the structured, role-based conversational guidance.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Effective communication and clear role distribution are universally recognized as the cornerstones of successful group collaboration. However, contemporary university learning management systems (such as eLeap from UNIMAS and UniMAP's E-Learning portal) primarily function as repositories for course materials rather than dynamic collaboration hubs. Because these platforms lack integrated tools to support structured, role-based coordination, student groups are frequently left to navigate teamwork without a formal framework. Consequently, collaborative assignments are routinely undermined by poor communication, fragmented workflows, and uneven involvement. Most critically, this lack of oversight fosters the pervasive issue of "free-riding," where certain individuals contribute minimally or not at all, leaving a fraction of the group to carry the academic workload.

This project implements a personalised chatbox in an educational website called Selab Management System from UNIMAS to guide the students based on their roles or the roles that they selected in the group. This chatbox also serves as a task reminder and progress updates to ensure that all the members are informed of their duties and deadlines. Guidances based on roles that this chatbox provides promote good coordination between group members and reduce the chances of miscommunication or task neglect. Moreover, this chatbox helps minimise the issue of free riding that always occurs during group assignments or projects by keeping each member accountable for their contribution, which encourages balanced participation and improves teamwork within the group.

Nowadays, using Artificial Intelligence tools and web-based learning systems is the key to improving communication and teamwork among students. According to Lin and Chang (2023), they stated that by encouraging Independent and engaged learning processes, implementing personalised chatbots in an educational setting can enhance student motivation, engagement, and results. The main goal of this project is to create a chatbox that is interactive and can adapt according to the user or the role that is selected by the user which will not only assist them in their group's work management but also help in learning engagement and avoid free riding issues by using TypeScript for the front-end development and AI API for smart responses.

1.2 Problem Statement

Most websites, especially in educational websites, are in the absence of personalised materials that can improve teamwork and role-based guidance. Students who work in groups without guidance usually struggle to meet deadlines, understand their own tasks, and communicate regularly with their group members. Thus, this leads to bad coordination within the team, uneven participation and the biggest issue, which is free riding, where the students give less contribution compared to others, in completing the assignment. Students can improve their teamwork and results by implementing a personalised chatbox which provides them with a task reminder, progress updates and role-based guidance that consistently helps or guides the students in completing their task. That way, students will be able to stay organised, communicate more efficiently and ensure that all students within the teams participate and are involved in their own projects.

1.3 Scope

This project focuses on developing a **web-based chatbox system** that:

- Integrates with SeLab Management System.
- Uses TypeScript for front-end functionality.
- Provides role-based communication and reminders (leader/member).
- Supports AI-generated or predefined responses through API integration.
- Covers only the student-side interface and chat functionalities (not lecturer dashboard).

1.4 Aims and Objectives

Aim:

To develop a role-based personalised chatbox that enhances teamwork, accountability, and learning engagement in educational websites.

Objectives:

- a. To analyse existing educational chat systems and their personalisation methods.
- b. To design and develop a chatbox that provides role-based communication and guidance.
- c. To test and evaluate the system's usability and functionality of the system developed.

1.5 Brief Methodology

This project implemented the Rapid Prototyping Model as the main development Methodology. This model highlights the iterative design, evaluation, and continuous

refinement based on the user comment or feedback to ensure that the final product or outcome satisfies both functional and usability requirements. This method works well for creating the Personalised Chatbox for Role-Based Guidance and Group Accountability since the system is very dependent on the user feedback and experimenting to improve the interaction quality and the role-based guidance effectiveness. Borsci et al. (2023) claim that by incorporating empirical validation and iterative improvements throughout the development process, software prototyping improves user-centred design and overall system usability.

This project starts with Need Analysis phase where the role-based communication is needed to be identified and the existing educational chatbots are to be reviewed. To guarantee a functioning and a positive experience for the users, the Design phase is phase where it is more focused on the chat interface, specifying role logic and organising user interaction flows. TypeScript, HTML and CSS are used to create the chatbox during the prototype and development process. AI APIs like OpenAI or Gemini are used in this project to give intelligent responses to the user. Node.js will be used by the backend automate task reminders and handle data. To confirm system performance, the validation phase is needed where it includes usability and functionality testing with a couple of student users as a sample. To determine the quality and happiness of interaction, Evaluation phase is conducted where the feedback from the students is gathered, the interaction quality is measured, and the chatbox is refined and improved for better user experience.

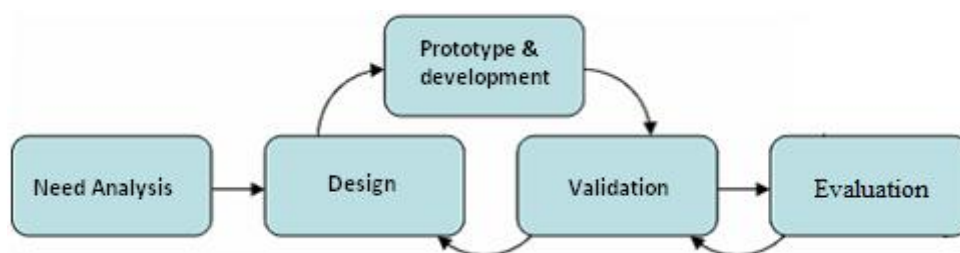


Figure 1.1: Rapid Prototyping Model (Alaoui et al., 2020)

1.6 Significance of the Project

This project helps promote fairness, responsibilities, and communicate more effectively in group activities such as project and assignment. It minimises the need of lecturer for task reminders, enhance the engagement between the students and helps the students adapt to their

role more efficiently. The system serves as the pillar for future smart learning assistants designed for online education.

According to Labadze, Grigolia, and Machaidze (2023), chatbox powered by AI can improve both teaching and learning by providing instantaneous support, personalised learning experiences and skill enhancement for students. According to their evaluation, these tools can save a lot of time especially for the lecturer and increase the learning effectiveness while also helping the students with their studies, fostering interaction between students and help improving their writing and problem-solving skill. By implementing these findings into this project, it corresponds to the current trend in educational technology, which the main priority are the student's motivation, personalised support and interactive learning environments

1.7 Expected Outcome

The project will produce a fully functional working personalised chatbox system that will be integrated into an educational website. The system will guide students through roles, remind them of tasks, update your progress and communicate through command-based interaction to coordinate group action and help them to answer. Furthermore, it is suggested that the proposed personalised chatbox will improve group members working together by promoting better interaction, communication, and better task distribution. Conversational interaction and automated reminders are also expected to decrease free-riding and enhance engagement on group assignments.

1.8 Summary

Finally, general background and motivation of the proposed personalised chatbox system for role based guidance and group accountability has been discussed in the end of this chapter. Group assignments are fraught with challenges for students, particularly communication, unclear roles, failure to deliver on time, and free-riding.

The system development was explained with a detailed description of the aims, objectives, scope, methodology, significance and expected outcomes of the project. The system of a personalised chat box is anticipated to facilitate team collaboration, accountability and communication among students by offering role-based directions and conversation support.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

This chapter reviews a few existing systems that can relate to the proposed personalised chatbox system. The purpose of this review is to understand how current tools support communication, task management and distribution, and collaboration among students in the same group, especially in educational settings. The strengths and weaknesses of the selected applications or systems are analysed to identify the holes that the proposed system to review. Relevant findings from recent AI chatbot studies are also integrated to show that modern educational tools can use personalisation and conversational interaction to improve engagement and accountability among group members.

2.2 Overview of Objective

The main objective of this chapter is to analyse the existing systems and research studies which are related to the development of personalised chatbox for role-based guidance and accountability, The goal is to better grasp weaknesses found in today's learning tools and then show how the new design might fill these missing parts.

This chapter examines existing educational chat systems and their approaches to personalisation. Platforms such as eLeap and Google Classroom primarily focus on content delivery and course management; however, they offer limited personalised communication features to support role distribution, task reminders, or structured collaboration within student groups. Identifying these limitations highlights the need for a personalised chatbot that can guide students based on their specific roles and responsibilities within collaborative learning environments.

This review also examines how reminders and progress updates improve teamwork. As mentioned in chapter 1, students who collaborate in teams usually have trouble remembering their deadlines, understanding their tasks, and maintaining consistent communication, which leads to uneven involvement and free riding. Reviewing current systems and AI chatbot studies provides insight into how automated reminders and progress notifications can enhance planning, engagement, and accountability in group work.

Finally, this chapter also reviews recent work on AI chatbots, and which focused on how they affect involvement, teamwork and knowledge gain. Recent studies also shown us that personalised AI chatbots can improve motivation, interaction or communication, and learning engagement especially with well-built systems using context-aware cues (Elkot et al., 2025; Kim & Lim, 2025). Such results highlight the value of an interactive chatbox that adapts to user's role and encourages balanced participation among group members.

2.3 Background

Group assignments are a major component of university learning, but students tend to encounter obstacles like unbalanced participation, overlooked duties or poor communication. Such problems often create uncertainty around who does what, late submissions, and free riding within group projects. To address these obstacles, the learning must not only deliver information but also provide structured support that support the students to stay organised and accountable. However, several existing educational platforms does not offer any personalised or role-based guidance, which reduced their ability to support teamwork effectively.

Recent research has highlighted the importance of personalised AI chatbots in improving the student engagement, learning motivation and communication. Elkot et al. (2025) found that AI chatbots with personalised feedback can enhance motivation and learning engagement, which proof that the benefits of systems that are able to adapt to individual needs. Likewise, Kim and Lim (2025) showed that role-sensitive and context-aware chatbot prompts can enhance the collaborative learning by improving the coordination and collective responsibility within a group. These findings show that personalised and adaptive chatbot systems have great potential to provide a support to both individual guidance and group accountability.

Furthermore, the SMARTA (Student Motivation and Reflective Training AI-Assistants) project examines how personalised AI chatbots can be incorporated into academic systems to act as a teachers or coaches. This study shows that structured conversational guidance, adaptive feedback and reflective questioning can enhance the student's motivation and engagement within the educational environment (Bieletzke, 2024). This study further reinforces the effectiveness of a personalised conversational systems in providing guidance in an educational setting.

Based on these findings, a personalised chatbox that deliver reminder, provides guidance which is based on the user's role, while boosting regular communication which can help in reducing free riding problem and promote a more balanced participation in group work. With a system like this ensures that each members understand their own responsibilities, stays updated on their assignments deadlines, and receives appropriate support from the personalised chatbox according to their role which ultimately improves their teamwork efficiency and overall learning outcomes.

2.4 System Continuation from Previous Work

This project is designed as the continuation of a previous system of a senior student, Wee Hong, who developed the SeLab Management System. The current system helps in managing the project and user types with basic group coordination of an educational environment.

Recent research on educational project management systems that highlight role-based collaboration and performance improvement also support this system structure, which includes many users and system support services. (Jali et al., 2026).

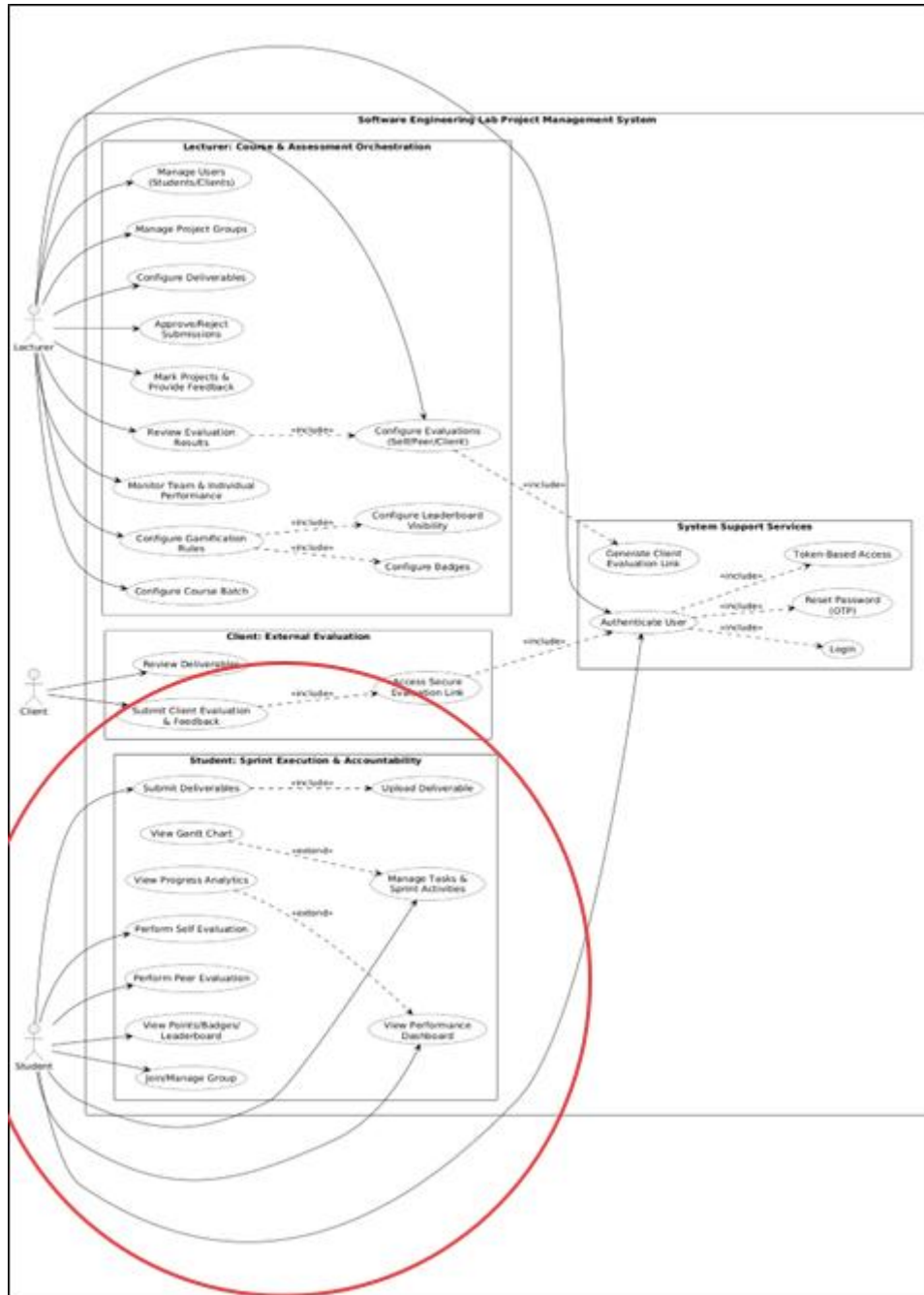


Figure 2.1: Use Case Diagram of the SeLab Management System

The use case diagram of SeLab Management System by showing different users and their system supports services like student, client, lecturer and those who provide support to the system. The diagram describes the system that helps project management, evaluation, and coordination of different users.

However, this project mainly focuses on the Student component specifically tasking, tracking progress and group accountability. The functionalities of lecturer and client are relatively unmodified so these are not discussed in detail.

The study does not seek to redevelop the whole system but to upgrade the existing platform with a personalised chatbox feature. Our personalised chatbox will offer you role-based guidance, task reminders, and interactive conversation to enable group accountability.

As a result, this project enhances the existing system to offer intelligent, conversational assistance. This will improve upon the role-based interaction. In addition, task coordination that exhibited gaps in previous chapters.

2.5 Existing Applications

This section discusses systems or applications which are commonly used for communication, organisation or joint work. These systems are selected due to the certain functionalities they share that are related to the goals of this project, such as task management, reminders or messaging capabilities.

2.5.1 Trello

Trello is a task management tools that helps the students to organise activities using boards, lists and cards. It is widely used in educational and professional settings for managing or overseeing projects and tracking progress.

Strength:

- Clear and structured visual layout
- Easy task categorisation
- Suitable for organising group assignments

Weakness:

- No personalised reminders
- No role-based communication
- No conversational or AI-powered interaction
- Does not enforce accountability among group members

Trello helps in project organisation, but it lacks the interactive communication and personalised guidance for students, which are essential in improving the participation and engagements of students especially in group settings.

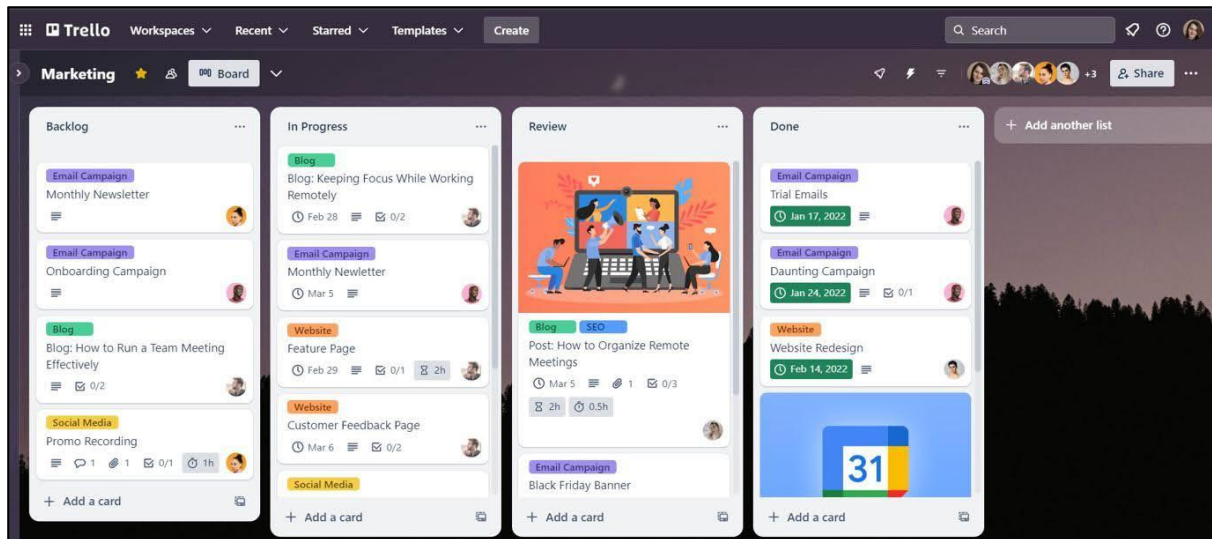


Figure 2.2: Existing System - Trello

2.5.2 Google Classroom

Google Classroom is an educational platform used to manage course materials, assign assignments to students, and facilitate communication between lecturers and students.

Strength:

- Strong integration with Google services
- Efficient assignment submission and announcements
- Easy for instructors to manage course activities

Weaknesses:

- Limited peer-to-peer collaboration features
- No automated reminders for group progress
- Does not provide personalised or role-based guidance
- Does not address free-riding issues in group tasks

While Google Classroom enhances the teaching process, it does not support the active collaboration or accountability among the students group.

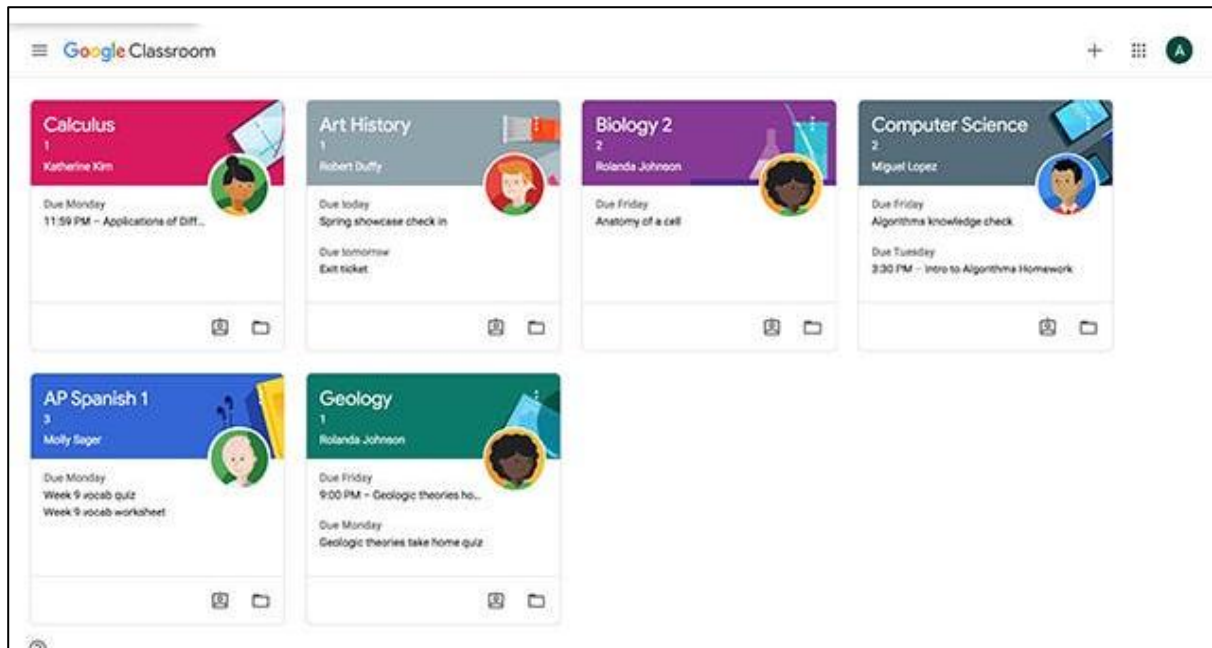


Figure 2.3: Existing System – Google Classroom

2.5.3 Jira

Jira is a tool used to track issues and manage projects, which are common among software teams. While one benefit includes task creation, another involves assigning duties to team members. Progress monitoring stands out, alongside workflow visuals via boards or sprint planners. Despite strengths in handling complicated initiatives, it lacks features tailored to personal messaging. Instead of supporting role-based guidance for learners during teamwork, its focus remains elsewhere.

Strength:

- Offers detailed task tracking and assignment features
- Provides workflow visualisation through Kanban or Scrum boards
- Supports team progress monitoring, deadlines, and issue reporting
- Highly customisable for different project requirements

Weakness:

- No personalised reminders based on individual roles
- Lacks conversational or interactive guidance for users
- Designed for professional project teams, not student learning environments
- Does not enforce accountability beyond manual task updates

While Jira works well for tracking software tasks, however it doesn't offer role-based messaging or smart prompts that help students to communicate better. This is because updates

must be done by hand, which can make it hard for users to stay involved, share progress, or take responsibility in team settings.

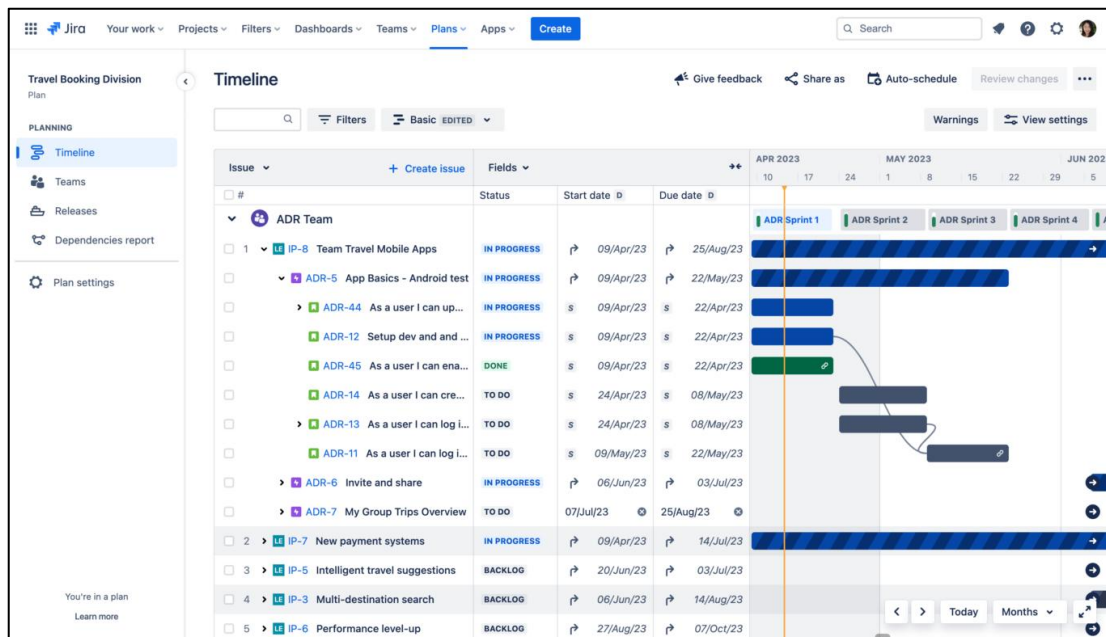


Figure 2.4: Existing System - Jira

2.5.4 Existing Application or System Comparison

The following table outline the comparison between the reviewed systems or applications:

Table 2.1: Comparison of Existing System

Application	Purpose	Strengths	Weakness	Supports Role-Based Guidance?
Trello	Project organisation	Clear structure, visual workflow	No reminders, no chat	No
Google Classroom	Learning management	Instructor-focused organisation	No collaboration for group tasks	No
Jira	Project and task tracking	Detailed task assignment, workflow visualisation	Designed for professional teams; no personalised or role-based communication	No

2.6 Review on Tools and Technology

This section reviews on the tools and technologies which are used specifically for the development and integration of the personalisation chatbox features. These tools support the implementation of role-based guidance, task reminders and accountability features that are within the chatbox.

2.6.1 Software

This part explains the programs used to build the custom chat system. This is because these tools support messaging features, they assist with interaction design. While handling user roles is necessary, it improves guidance accuracy. Although storing information matters, proper data control ensures reliability. As functionality depends on integration, each component is needed to work smoothly together.

2.6.1.1 TypeScript

TypeScript extends JavaScript through static typing. For this work, it handled the custom chatbox logic such as user roles in messages, reminders, or managing data. Using it helps reduce errors and makes updates easier. This is important to build a structured and scalable chat-based systems.

2.6.1.2 ReactJS

ReactJS helps in creating user interfaces. For this project, ReactJS was used to develop the chatbox user interface which enables dynamic message display and real-time interaction. Its component-based architecture which allows the chatbox to update efficiently based on the user roles and the system responses.

2.6.1.3 NodeJS and ExpressJS

NodeJS is a server-side runtime environment and on the other hand which is ExpressJS is a web framework which is built on NodeJS. In this project, both NodeJS and ExpressJS were used to implement backend services for the personalised chatbox. These services can handle the chat communication, role retrieval, reminder logic and data exchange between and the database.

2.6.1.4 MongoDB

MongoDB is a non-relational database which is used for storing application data. Within this project, MongoDB is used to store information related to the personalised chatbox such as the user's role, chat messages, task-related updates and the reminder records. Its flexible data structure also support the efficient handling of chat-based interactions.

2.6.1.5 Redis

Redis is an in-memory data store used to support fast data access. In this project, Redis is used to cache frequently accessed data and manage temporary information related to reminders and chat interactions, thereby improving system performance.

2.6.1.6 Docker (Docker Desktop)

Docker lets apps run in separate spaces using a containerisation platform. For this project, Docker Desktop is used to manage and run the personalised chatbox system during development. Docker also helps in ensuring consistency across development environments and simplifies system configuration.

2.6.2 Hardware

The personalised chatbox system does not need any specialised hardware components. The development and testing are carried out using a standard computing devices such as laptops or desktop computers with internet access.

2.7 Summary

In conclusion, the current systems that are being reviewed or examined reveal that while Trello, Google Classroom, and Jira provide good organisational support to users, they lack personalised role-based coaching or guidance, accountability features, and tailored reminders required for successful student group work. These gaps are supported by research as well which is Stöhr et al. (2024), which shows that the students actively utilise AI chatbots and find them very helpful and Elkot et al. (2025) which highlighted that AI chatbots can improve learning outcomes through customised guidance, while their effect depends on a correct system architecture. Chatbots with role-sensitive and organised prompts can improve group efficacy and coordination which involve collaboration between group members (Kim & Lim, 2025). These results highlight the need for an integrated solution that provides role-based support, automatic reminders, and personalised communication within a single chat interface, since the current systems do not encourage cooperation or balanced participation.

CHAPTER 3: METHODOLOGY

3.1 Overview

This chapter further explains the methodology which is used to develop the proposed personalised chatbox system. The main purpose of this chapter is to outline the system's planning, analysis, design, development and evaluation to make sure that role-based guidance and group accountability are effectively supported in an educational environment. This project's development approach, system analysis, prototype activities and the implementation process which are described in more detail in the chosen methodology. This chapter illustrates how continuous improvement and the feedback that are integrated to produce a functional and reliable AI chatbox system help solve communication gaps, task coordination issues and free-riding issues, which were found in previous chapters, by using an iterative and user-centred development model.

3.2 Rapid Prototyping Model

The Rapid Prototyping Model is a method for developing systems that focuses on rapidly producing working prototypes, assessing them with users and improving the systems through constant feedback. This model works really well for interactive systems where the success of the final product depends heavily on user experience and system usability. In this context, software prototyping is the technique of creating and testing the early version of the system, which is an essential component of the Rapid Prototyping Model. Borsci et al. (2023) claim that by helping the developers to find the usability problems and design flaws ahead of time to complete the system deployment, this method supports the early validation of the system design and functionality.

In this project, the Rapid Prototyping Model is used to create the personalised chatbox for role-based guidance and group accountability. The proposed system requires regular evaluation of chat interactions, role-based responses, and reminder effectiveness, which necessitated an iterative development approach. Through rapid prototyping, the early versions of the personalised chatbox interface and the message flow which can be tested to evaluate the clarity, responsiveness and usefulness from the user's stance.

In general, the Rapid Prototyping Model supports the continuous improvement of the personalised chatbox system in the education environment by integrating user feedback through the development process. This technique guarantees that the final system is user-

centred, reliable, and capable of supporting effective communication, accountability, and collaboration among students in groups in an educational environment.

3.2.1 Planning

The first phase in the Rapid Prototyping Model is planning, where the problem statement, objectives and overall direction of the proposed systems are clearly defined. These days, especially in group work in educational settings, students tend to face similar challenges such as poor communication, unclear task responsibilities, missing deadlines, and free riding among group members. The current educational platforms do not provide adequate personalised or role-based guidance in order to tackle the issues. Thus, the main purpose of creating the personalised chatbox system is to design and implement a chat-based solution that helps in role-based guidance, task reminders and group accountability in an educational platform.

The identification of the system requirements is a very critical activity in this planning phase. The problem analysis in Chapter 1 and the examination of the current systems in Chapter 2 helps as the foundation for the system requirements. These requirements include role identification such as leader and member, personalised chat which response according to the user, task and deadline reminders and an interactive chat interface. User requirements are also collected through surveys provided to people, especially students, to better identify common problems within group projects and the expectations for chat-based support.

Additionally, the scope of the proposed system is clearly stated during this phase. Because of the time and the limitations of the project, the system focuses only on the student-side features for now, such as chat interaction, role-based guidance and reminders. The lecturer-side management and advanced administrative features, as of now, are excluded from this project. A project schedule is also prepared using a Gantt chart to ensure that development activities are properly planned and executed within the Final Year Project timeline.

3.2.2 Analysis

In the second phase of the Rapid Prototyping Model, a rigorous comparison is made between the current methods used in educational group work and the proposed personalised chatbox system to demonstrate the need for its creation. This analysis goal's is to explain the limits of

the current educational platform in providing role-based guidance, task coordination and accountability which tend to lead to poor communication and free riding issues within a group. During this phase, the proposed system's user, software, and hardware requirements are also assessed to ensure alignment with the project objectives. The process of identifying user requirements is expected to be carried out using questionnaires, in which students who are the users are asked to share their experiences, issues, and any expectations related to group assignments and chat-based guidance.

3.2.2.1 Analysis of the Current Method

In the current educational environment, efficient communication and coordination within a group are important for successfully completing group work. Nonetheless, most current educational platforms depend primarily on human communication methods such as messaging apps, email or simple announcement tools. These systems tend to require students to actively seek information, remember their work deadlines and manually track their own progress which can be wasteful and inconsistent.

Additionally, due to the current systems that lack role-based guidance, students tend to struggle with their particular roles within a group. Leaders usually bear the responsibility of monitoring their group's progress and reminding the members of their task, whereas other members may become inactive or disengaged. This situation typically results in uneven contributions and free riding, with members giving less or failing to complete their assigned responsibilities within the given time.

Furthermore, the current platform fails to provide automated or personalised reminders that can adapt according to the user's role or task status. As a result, missing deadlines, poor task coordination and last-minute work are typical issues within group projects. Individual contributions are sometimes not clearly visible, making it difficult to hold members accountable. These limitations highlight the need for a more interactive and personalised system which can support role-based communication and accountability in student group projects.

3.2.2.2 Analysis on Proposed Application

The proposed personalised chatbox system has been specifically created and developed to resolve the limitations which are found in the current approach for managing the students' group activity. The main goal of creating this system is to ensure a more effective communication, role-based guidance, and accountability within a group using an interactive chat-based platform. By integrating the chatbox into an educational website, students can be constantly guided and reminded of their responsibilities based on their assigned roles within a group.

The personalised chatbox support role-sensitive interactions, with leaders receiving prompts regarding task monitoring and progress follow-ups, and members getting reminders and guidance on their assigned tasks and deadlines. Furthermore, the systems help in automated reminders and progress updates, which help in eliminating missed deadlines and increasing task coordination. Conversational participation also encourages the students to remain engaged, communicate much more frequently, and contribute more consistently to group activities.

The proposed system distinguishes between two user types: group leaders and group members. Each user type also has their own responsibility and access to the system functionality.

Group Leader can perform the following functions:

- a) Login to the SeLab educational system
- b) Create and manage his group tasks
- c) Assign tasks to his group members
- d) View task progress and completion status
- e) Receive reminders to follow up with group members
- f) Communicate with members through the personalised chatbox

Group Member can perform the following functions:

- a) Login to the SeLab educational system
- b) View their assigned tasks and deadlines
- c) Update task progress

- d) Receive personalised reminders and guidance
- e) Communicate with the group leader through the chatbox

The proposed personalised chatbox approach which support balanced participation, eliminates the free-riding behaviour and increases overall team efficiency via role-based engagement and automated guidance. This technique makes sure that the students are well informed of their duties while receiving the continuous guidance using a simple and interactive chat interface.

3.2.2.3 Questionnaires

Questionnaires were distributed to a group of target users to collect user requirements, habits, and opinions regarding the proposed AI-powered, personalised chatbox for role-based guidance and group accountability. By conducting questionnaires for the target users, the developer can gain a clearer and more detailed understanding of users' perspectives on group projects, their AI chatbot consumption rate, and their perceptions of the proposed system.

The questionnaires were randomly distributed to 47 respondents at Universiti Malaysia Sarawak (UNIMAS). Participation was voluntary and anonymous, and respondents were informed that they could withdraw at any time without penalty. All respondents must provide consent before proceeding with the questionnaire.

The questionnaire consists of three sections. Section A captures respondents' demographic information, Section B captures respondents' habits and perceptions related to chatbot usage and group projects, and Section C captures respondents' perceptions toward the proposed AI-powered personalised chatbox system.

Section A: Demographic Profile

2. Age

47 responses

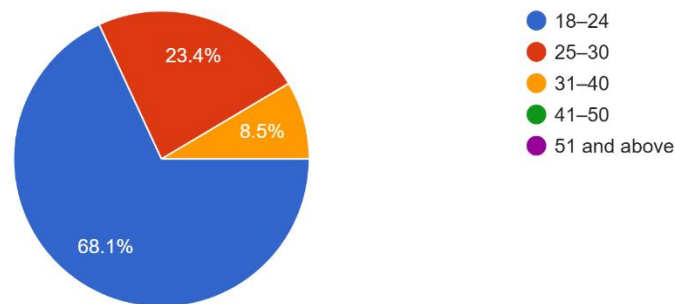


Figure 3.1: Age Distribution of Respondents

The age distribution shows that most of the respondents are within the age group of 18-24 (68.1%), followed by 25-30 (23.4%) and 31-40 (8.5%). No respondents were recorded as being over 40, indicating that most respondents are within the university student age range.

3. Gender

47 responses

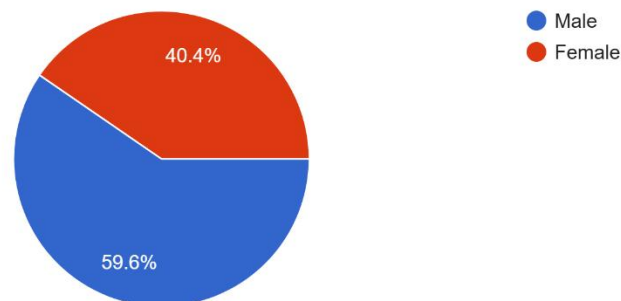


Figure 3.2 : Gender Distribution of Respondents

The gender distribution shows that 59.6% of respondents are male and 40.4% are female. This proportionally balanced distribution assures that both genders are represented within this study

4. Academic Status

47 responses

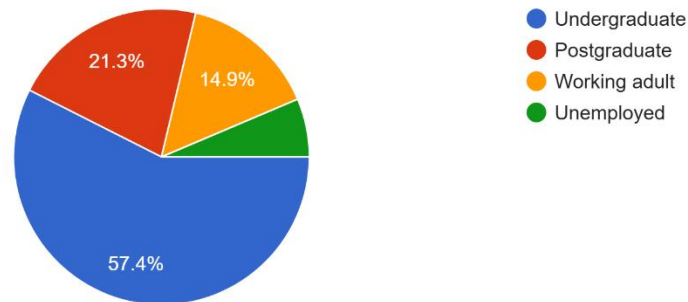


Figure 3.3: Academic Status of Respondents

Most of the respondents are undergraduate students (57.4%), followed by postgraduate students (21.3%), working adults (14.9%), and a very small number of unemployed respondents. This confirms that the respondents are actively involved in academic environments where group projects are common.

5. Have you used AI chatbots before?

47 responses

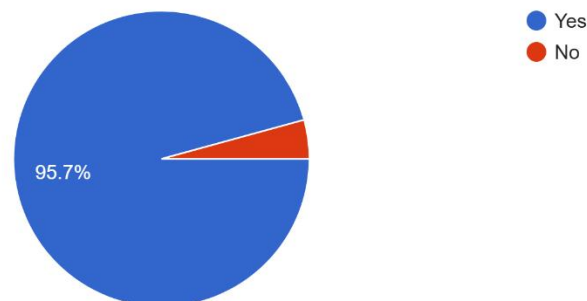


Figure 3.4: Experience with AI Chatbots

The results show that 95.7% of respondents have previously used AI chatbots, which indicates a very high-level of familiarity with AI-based conversational systems among the target users.

Section B: User Habits and Perceptions

6. How often do you use chatbots (ChatGPT, Gemini, etc)

47 responses

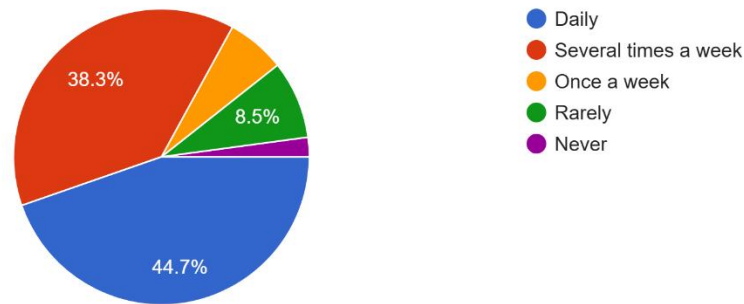


Figure 3.5: Frequency of Chatbot Usage

Most respondents reportedly use chatbots daily or several times a week, which suggests that chatbot usage is already integrated into their academic and daily routines.

7. For what purpose do you usually use chatbots? (Tick all that apply)

47 responses

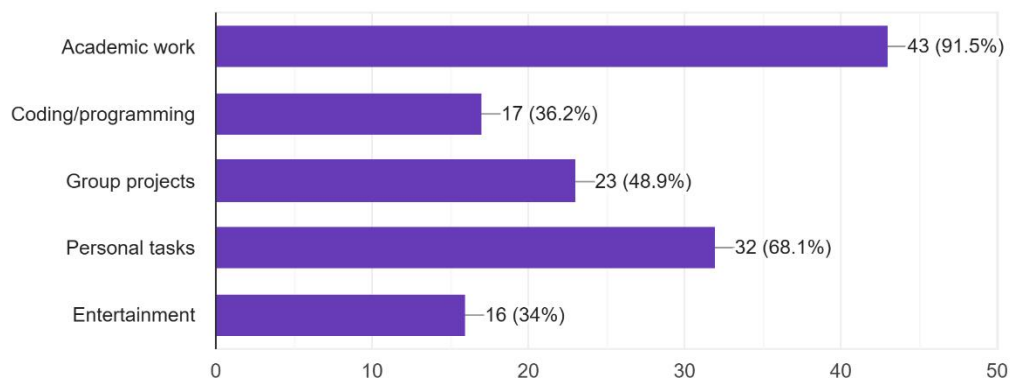


Figure 3.6: Purpose of Chatbot Usage

Respondents primarily use chatbots for academic work, followed by personal tasks, group projects, coding or programming, and entertainment, which highlights the use of chatbot technology in educational contexts.

8. How often do you participate in group projects?

47 responses

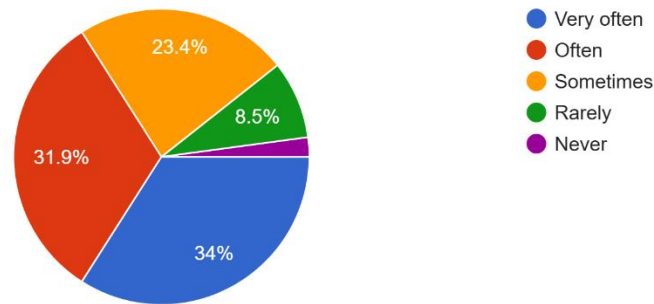


Figure 3.7: Frequency of Group Project Participation

Most respondents participate in group projects very often or often, which indicates that collaborative work is a significant component of their academic experience.

9. Do you normally face problems in group communication or accountability?

47 responses

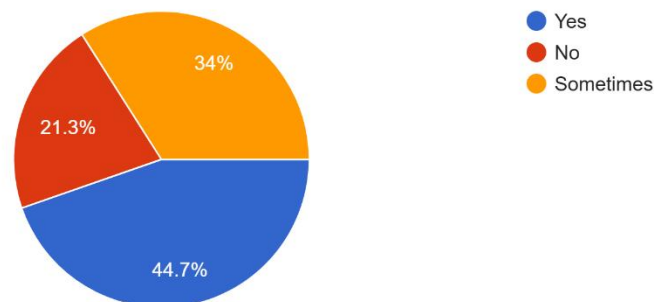


Figure 3.8: Issues in Group Communication and Accountability

A high number of respondents are reportedly that they sometimes or often face problems related to group communication or accountability, which highlights common issues in managing group work.

10. In group projects, how often do members complete their assigned tasks on time?
47 responses

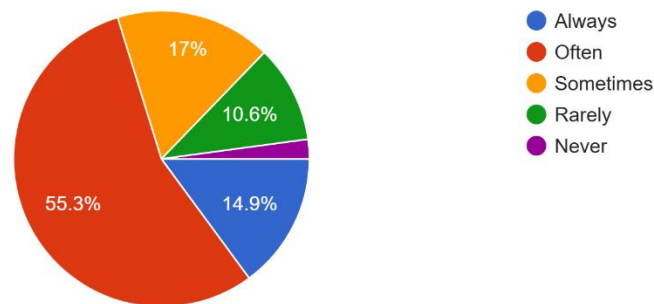


Figure 3.9: Task Completion Timeliness in Group Projects

While many respondents indicated that tasks are often completed on time, delays still occur, which suggests the inconsistency in task completion within a group.

11. How often do you feel you must remind group members about deadlines or tasks?
47 responses

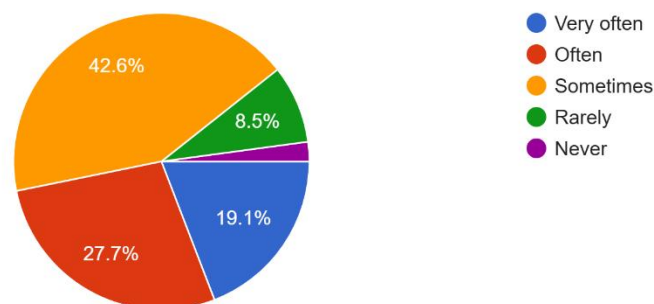


Figure 3.10: Frequency of Task and Deadline Reminders

Most respondents stated they always need to remind their group members of tasks or deadlines, indicating the need for automated reminder support.

12. How effective is your group at keeping track of each member's responsibilities?
47 responses

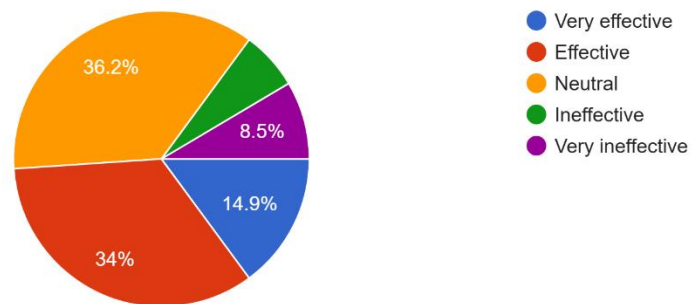


Figure 3.11: Effectiveness of Responsibility Tracking

Respondents' perspective on how effectively their groups can keep track of individual responsibilities was mixed, with many selecting neutral or ineffective, indicating that there is room for improvement.

Section C: Perception Toward the Proposed AI-Powered Personalised Chatbox

13. A AI-Powered Personalised chatbox could help improve communication during group projects.
47 responses

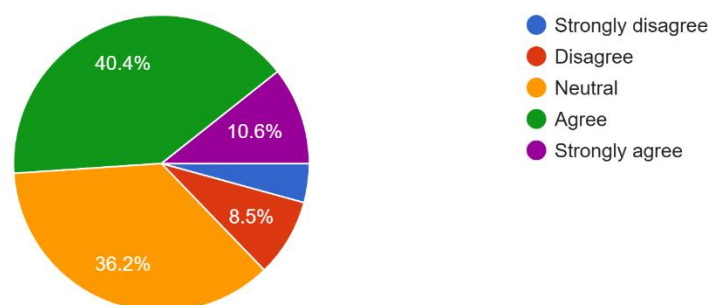


Figure 3.12: Perceived Improvement in Group Communication

Most respondents agreed or strongly agreed that an AI-powered personalised chatbox could improve communication during group projects.

14. An AI chatbox that gives different guidance based on roles (Leader/Member) would be beneficial.

47 responses

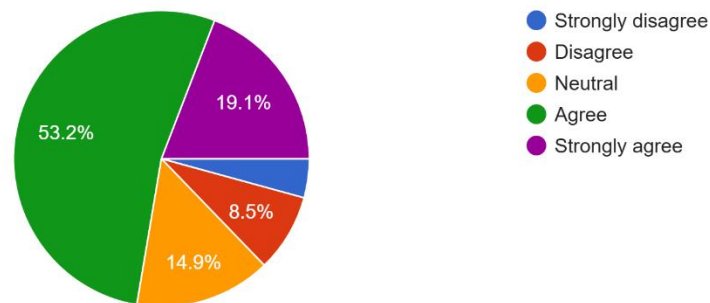


Figure 3.13: Perception of Role-Based Guidance

Respondents showed a very positive perception toward role-based guidance, indicating that different guidance for group leaders and members would be beneficial.

15. Having an AI chatbox remind members about tasks and deadlines would be useful.

47 responses

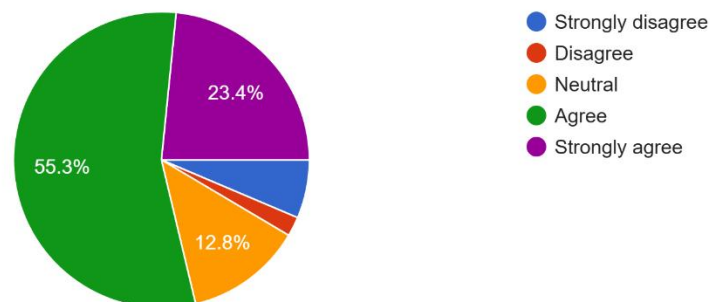


Figure 3.14: Usefulness of Task and Deadline Reminders

A large proportion of respondents agreed that the automated reminders would be useful for managing the tasks and deadlines.

16. An AI chatbox could help reduce situations where group members forget their responsibilities.
47 responses

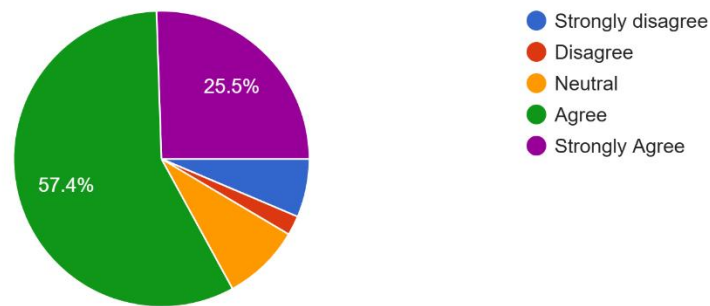


Figure 3.15: Reduction of Forgotten Responsibilities

Many respondents believed that the proposed system could help reduce situations where group members forget their responsibilities.

17. A system that tracks progress and prompts members could increase accountability in group projects.
47 responses

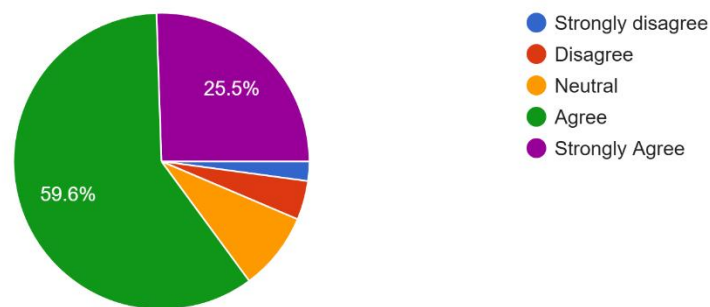


Figure 3.16: Perceived Increase in Accountability

Respondents admit that tracking progress and prompting members could increase accountability in group projects.

18. I believe such a system can make group work more organized.

47 responses

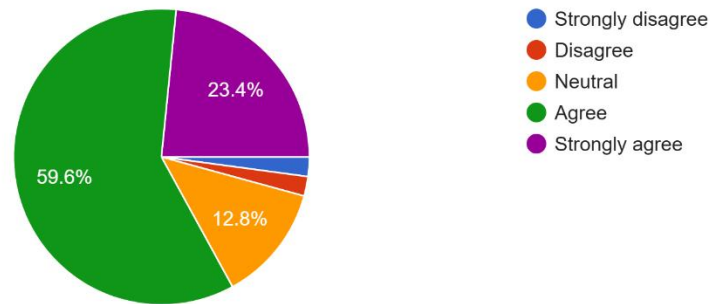


Figure 3.17: Organisation of Group Work

Most respondents trust that the proposed chatbox system could make group work much more organised.

19. Using this chatbox in an educational platform can help create a more accountable group work environment.

47 responses

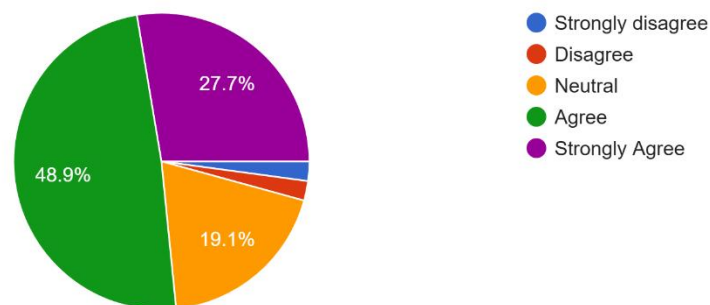


Figure 3.18: Accountable Group Work Environment

These responses indicate that most respondents agreed or strongly agreed that using the proposed chatbox in an educational setting can help provide a more accountable group work environment. This suggests that features such as progress tracking, reminders, and role-based guidance can improve responsibility and accountability among group members.

20. I think using this chatbox can make the distribution of group responsibilities more fair and transparent.

47 responses

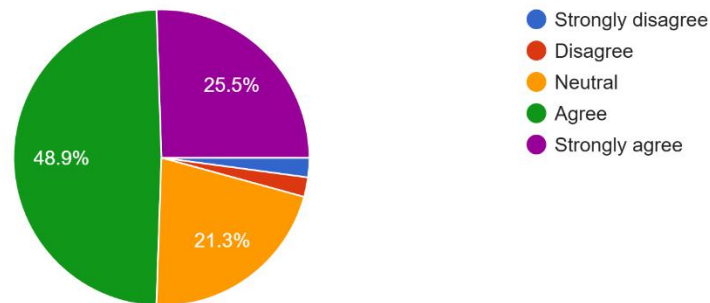


Figure 3.19: Fairness and Transparency of Responsibility Distribution

The results indicate that most respondents agreed (48.9%) or strongly agreed (25.5%) that using the proposed chatbox can make the distribution of group responsibilities fairer and more transparent in group works. This suggests that respondents perceive the system as someone who is capable of reducing unequal workload distribution by improving task visibility and role clarity within group members. The presence of neutral responses shows that some respondents may need further experience with the system to fully give more concrete feedback on its effectiveness.

An open-ended question was also included as question 21 to explore the suggestion of respondent for features for an AI-powered chatbox designed to support group projects and accountability. In general, respondents highlight features related to task and deadline management, shared calendars, progress tracking, fair workload distribution, and automated reminders. More suggestions include the file sharing, centralised resource management, discussion and meeting summaries, brainstorming assistance, and an easy-to-use, human-centered interface that encourages participation without any pressure toward the user. These qualitative responses provide valuable guidance for the design of the proposed personalised chatbox system.

3.2.2.4 Software Requirements

Software requirements for the proposed personalised chatbox system had been discussed extensively in section 2.5 in Review on Tools and Technology in Chapter 2. Below is the summary of software that will be used in creating the proposed personalised chatbox system.

Table 3.1: Software requirements for the proposed personalised chatbox system

Software	Description
TypeScript	TypeScript is a strongly typed programming language that builds on JavaScript. This software is used in this personalised to carry out the chatbox logic, role-based guidance handling, and reminder functionality while reducing runtime errors and improving code maintainability.
ReactJS	ReactJs is a front-end JavaScript library used in web development to build interactive elements on website User Interfaces. In my personalised chatbox, a JavaScript library is used to develop the chatbox user interface. ReactJS can provide dynamic message rendering and real-time interaction, which makes it suitable for building an engaging chat-based system.
NodeJS and ExpressJS	NodeJS is a server-side runtime environment and on the other hand which is ExpressJS is a web framework which is built on NodeJS. These tools are used to handle backend logic of my personalised chatbox such as chat communication, role verification, reminder processing, and the data that is being exchange between frontend and backend.
MongoDB	A NoSQL database used to store user information, user roles, chat messages, task assignments given by group leader, and reminder records. Its flexible schema that provides the dynamic nature of chat-based interactions.
Redis	Redis is an in-memory data store used to support fast data access, such as scheduled reminders and frequently accessed chat-related information, to improve system performance.
Docker (Docker Desktop)	A containerisation platform that is used to manage the development environment in my project. Docker also helps in ensuring consistency across development environments and simplifies system configuration.
Google Chrome	Google Chrome web browser is often used in this proposed personalised chatbox development to search for related information. Moreover, it is used to test the interface and the functions of the proposed personalised chatbox.
Windows Operating	The operating system used for developing and testing the proposed system due to its compatibility with the selected development tools and widespread usage.

3.2.2.5 Hardware Requirements

The proposed personalised chatbox system does not require any hardware requirements for development and implementation. The system itself can be developed and tested using a standard personal computer or laptop with enough processing power, memory, and storage to

support the selected development tools and software platforms. Basic hardware components which include a processor, random access memory (RAM), storage device, and good internet connection are sufficient to run the development environment, test the chatbox features, and deploy the system successfully without facing any obstacle.

3.2.2.6 Design and Prototyping (UML)

The design and prototyping phase emphasize on the organising the system requirements into a more clear and structured design before the development starts. This phase makes sure that the personalised chatbox system can support the role-based guidance and group accountability by defining the user roles, the system functions and the engagement in an organised manner.

Unified Modeling Language (UML) is used to represent the functional behaviour of the system. A use case diagram is used to illustrate the relationship between group leaders and group members using chatbox for tasks such as communication, the task management, and receiving continuous reminders. This diagram help provides a high level of the system capabilities and helps confirm the system requirements before prototype is deployed.

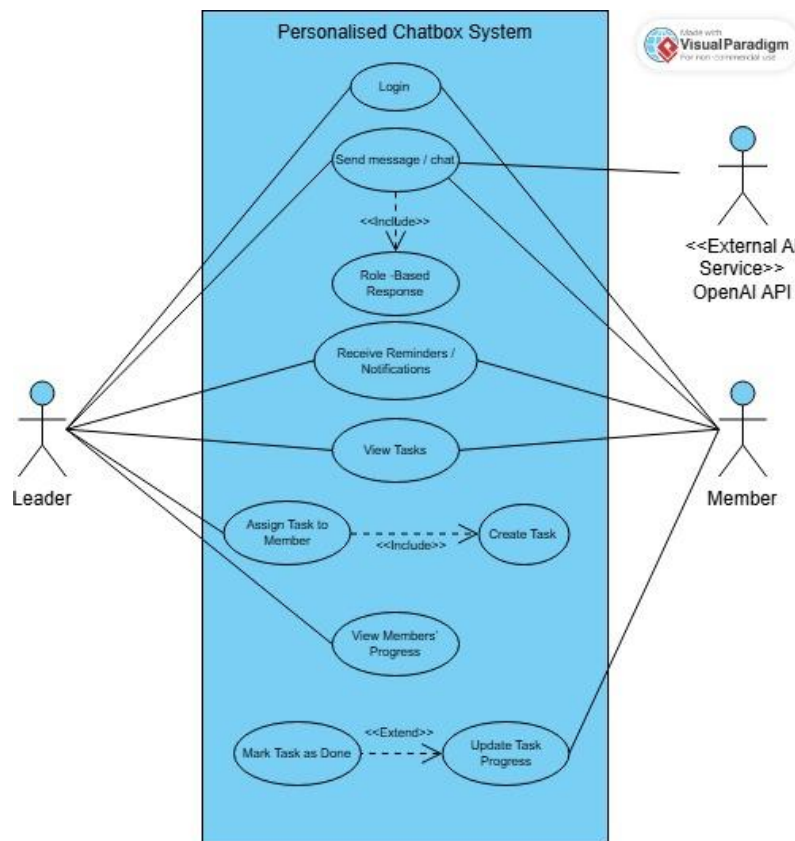


Figure 3.20: Use Case Diagram

UML is widely used in software engineering to model system requirements and the user-system engagement due to its ability to provide clear, standardised and technology-independent representations. In particular, use case diagram are very effective in transforming the user stories and functional requirements into visual models that capture the interactions between the actors and system functionality, facilitating early validation and communication of the system behaviour (Molla et al., 2024). The UML use case diagram illustrated in figure 3.20 concludes the functional behaviour of the personalised chatbox system, which illustrates how group leaders and group members interact with key system operations such as communication, task management, and automated reminder throughout the prototyping and implementation stages.

3.2.2.7 Wireframes

Wireframes are used to provide a simple visual representation of the user interface layout before much more detailed development starts. In this project, the personalised chatbox system is design as an incorporated feature within the current educational website rather than as an independent interface. Thus, the wireframe mainly focuses on the chatbox setting and its interactive features, without changing the general website structure.

The wireframe is meant to give a conceptual picture of how users interact with the personalised chatbox rather than to describe the detailed visual design features. Instead of showing the detailed visual or aesthetic elements, the wireframes demonstrate the functional behaviour of the chatbox, particularly in supporting communication, task awareness, and reminders within group-based activities.

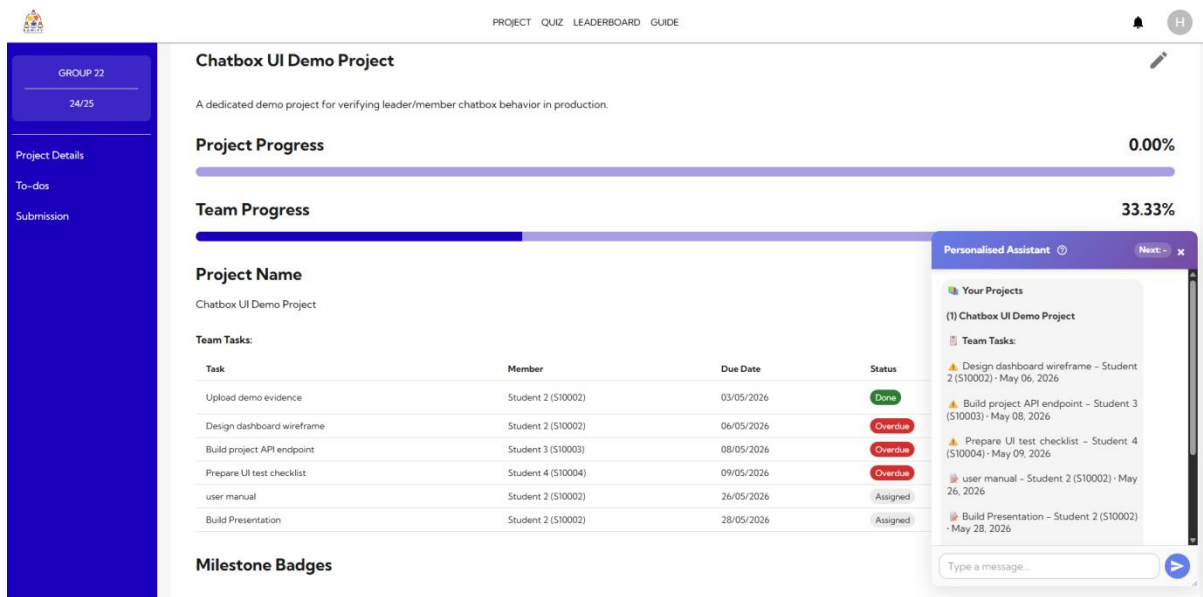


Figure 3.21: Wireframe of the Personalised Chatbox Interface (Leader Initial View)

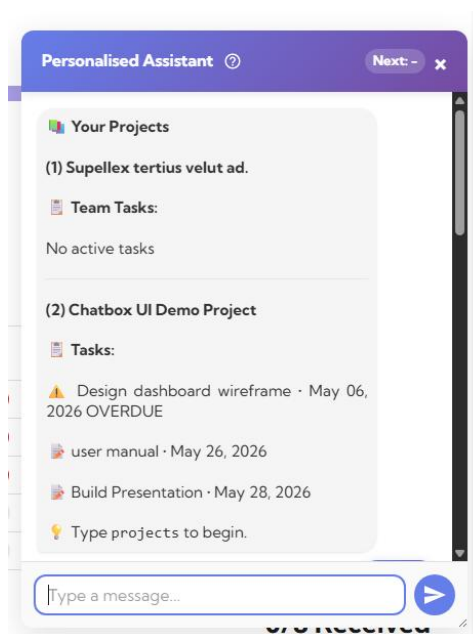


Figure 3.22: Wireframe of the Personalised Chatbox Interface (Member Initial View)

Figure 3.21 and 3.22 presents the general layout of the personalised chatbox integrated into the project page. Figure 3.21 illustrates the initial view for a group leader, while Figure 3.22 shows the initial view for a group member. both chat display section lets the group leaders and group members to view the conversation history, while the entry field and send button help the user to interact with the system itself. The reminder section speaks for the automated alerts which is about the tasks and deadlines. As the personalised chatbox system is developed using the Rapid Prototyping Model, the chatbox is repeatedly refined during the development phase and the final interface design is presented in Chapter 4.

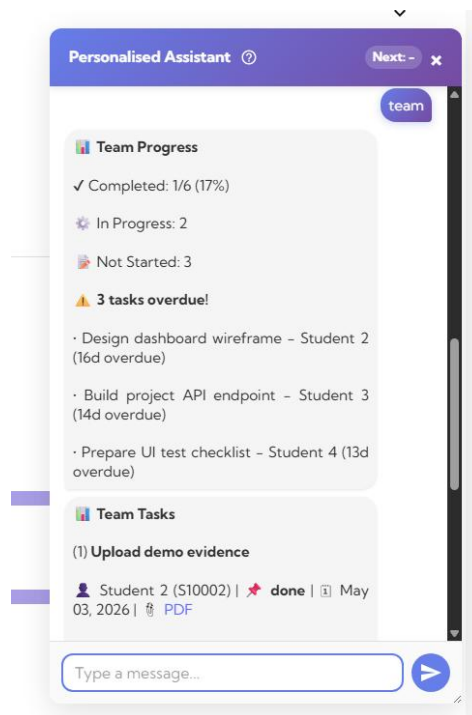


Figure 3.23: Wireframe of Leader Task Management via Personalised Chatbox

Figure 3.23 illustrates the interaction between the personalised chatbox and leader where the leader uses the personalised chatbox to monitor and manage group tasks. In this view, the leader receives all information about their group members such as overall task completion progress and the status of tasks assigned to the individual group members through the chatbox. The chatbox system responds by showing structured task status and reminder messages within the chat interface. This wireframe demonstrates how task monitoring and accountability functions are supported through conversational engagement, without introducing additional any task management interfaces.

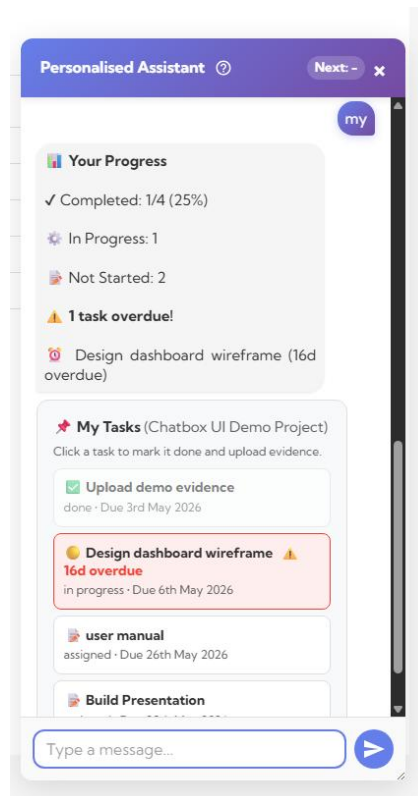


Figure 3.24: Wireframe of Member Task and Reminder View via Personalised Chatbox

Figure 3.24 shows the interaction between group members and the personalised chatbox system. In this view, the chatbox displays the tasks that were assigned by the group leader, along with their current status, their progress and due dates. Reminder messages are also presented to inform the group member of their responsibilities. This wireframe highlights how the personalised chatbox provides support for individual task awareness, and responsibility management with a conversational interface, which enables group members to track their assigned tasks effectively within the same system environment.

3.2.3 Prototype Cycle

The Rapid Prototyping Model's third phase, which is the prototype cycle, involves creating, testing, and improving the system prototypes iteratively until the identified user requirements are satisfied. In this phase, the early iterations of the personalised chatbox system are continuously refined through assessment and feedback. To make sure that the system meets the usability standards and successfully provides the necessary support for role-based guidance and group accountability, communication between the developer and the users is important.

3.2.3.1 Develop

Based on the requirements which are found during the analysis phase, the early prototypes of the personalised chatbox system which are created immediately during the development stage. A working prototype is built by setting up the essential features such as chat interaction, role-based guidance, task reminders, and the backend communication. While frontend interface designed for the users to interact with the personalised chatbox, the backend handles the data processing, role verification and message handling between the user and the chat.

User interface design principles like simplicity, consistency, visibility and feedback are used to make sure that the usability and the clarity during the interaction. Once the prototype is developed, the system logic and code structure are examined to reduce the errors before going to the demonstration stage.

3.2.3.2 Demonstrate

In this stage, the developed prototype is demonstrated to the users to received evaluation and feedback from them. The prototype is tested to make sure that it satisfies the project objectives and that all the important features are working as intended. During the demonstration. Users engage with the chatbox to evaluate the role-based responses, reminder effectiveness and the overall usability.

Throughout the demonstration, the user feedback and the comments given by the users are recorded and studied. This feedback is crucial in finding the areas that needed improvement and guides further refinement of the system in the following stage of the prototype cycle.

3.2.3.3 Refine

The demonstrated prototype continues to the refinement stage if it below the user expectation or the systems objective. At this stage, the improvement is made to the personalised chatbox interface, and the system features according to the user feedback. To enhance the system performance and the user experience, any usability issue, the logical weaknesses, or the feature limits are addressed properly.

The refined prototype is then demonstrated again for more evaluation. This iterative cycle of development, demonstration and the refinement will continue until the personalised chatbox systems satisfy all the identified requirements and achieve much more satisfactory usability

and functionality. Once the prototype is accepted, the system will proceed to the following phase for stage of testing.

3.2.4 Testing

In the testing phase, the personalised chatbox system is examined properly against the identified requirements to make sure that it is functioning correctly and support the role-based guidance and group accountability as intended. Usability testing is carried out to evaluate the clarity of the responses and the effectiveness of the user engagement, while the functional testing is also carried out to guarantee important aspects including chat interactions, role identification, task handling and reminder delivery. Feedback which are gathered during the testing is recorded and used to improve the system reliability and the user experience before the deployment.

3.2.5 Implementing

During the implementation phase, the completed personalised chatbox system is integrated in an educational website setting. At this stage, the system is configured to manage the real user interactions, with the backend services and the database components supporting the real-time data processing. To guarantee consistent system performance and successful operation in an actual educational setting, basic maintenance tasks, such as small enhancements and bug corrections, may be executed.

3.3 Summary

Chapter 3 discusses about Rapid Prototyping Model, which is the chosen methodology used to develop the proposed personalised chatbox for role-based guidance and group accountability. Every phase of the methodology has been comprehensively described in this chapter as well as explaining how the methodology is applied to construct the proposed personalised chatbox.

In the planning phase, active participation between the developer and the users which is needed to identify the problem statements and the objectives of the proposed personalised chatbox system. Besides, user requirements are collected in this phase by distributing questionnaires to a group of target users. Thus, a complete analysis is carried out to analyse all the requirements that are supportive to the proposed personalised chatbox development. The logical design of the system is described by using a Unified Modeling Language (UML) and structured design concepts to represent the system interactions and the architecture, while

wireframes are used in this proposed personalised chatbox to illustrate the chatbox interface layout. This chapter also describes the prototype cycle which consists of developing demonstrating and refining. Finally, testing and implementation which are discussed to assures that the personalised chatbox system operates properly and is ready to be deployed in real-world educational website settings.

CHAPTER 4: IMPLEMENTATION

4.1 Overview

This chapter describes the implementation of the proposed chatbox system for role-based guidance and group accountability. Physical and logical implementation are used to describe the development, which is based on the analysis and design which are covered in chapter 3. While the logical implementation emphasises system features like the task management, role-based guidance and also conversational interaction between the system and the student. In addition, the tools and the technologies which are used, as well as the integration of the frontend, backend and the database are reviewed.

4.2 Installation and Configuration of System's Components

To start developing the proposed chatbox, some software was required to be installed and configured in advanced.

4.2.1 React

React is a JavaScript library used to generate user interfaces. It is usually used in current web development due to its component-based design and the ability to generate a dynamic and interactive applications. In this project, React is used in order to create the frontend for the suggested personalised chatbox system. It helps display the chatbox interface, accepts any user input, and displays real-time changes for example messages, task status, and reminders.

To begin, React requires Node.js and npm to function properly in the development environment. These tools allow the developers to manage their project dependencies and run their applications or system locally. After installing Node.js, use the following command to install the React project dependencies: `npm install`

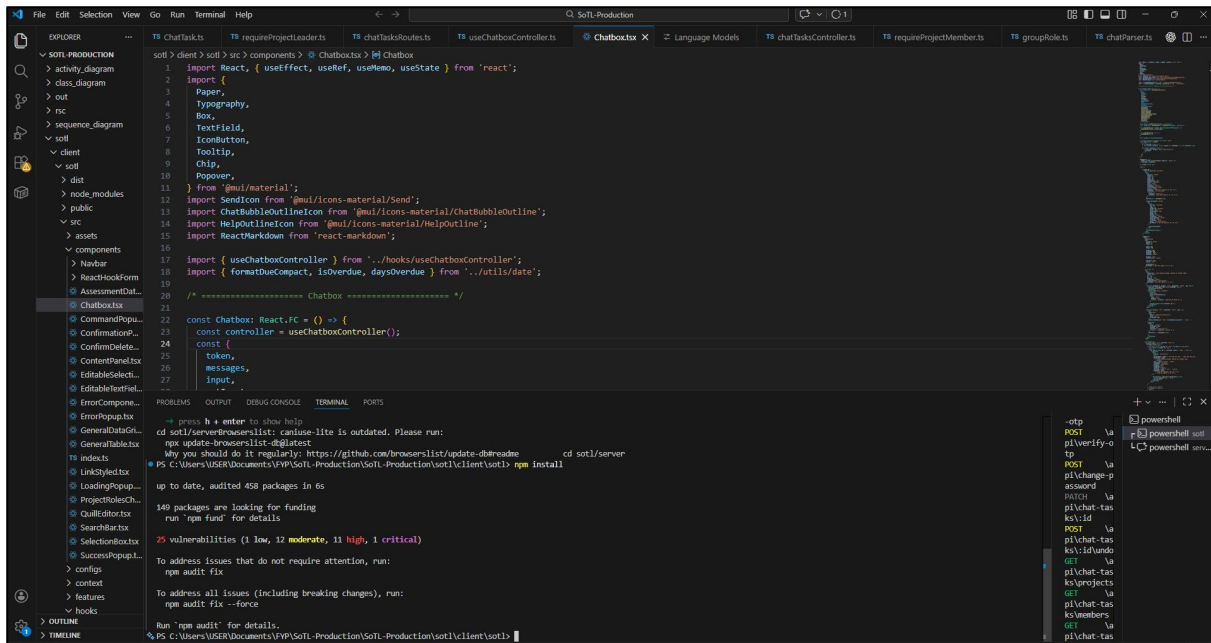


Figure 4.1: Installation of project dependencies using npm in the terminal

After the installation is completed, the React application can be executed using the development server with the following command: `npm run dev`

This command starts the program in development mode and allows the system to be observed through a local browser. Figure 4.2 shows the React application running properly using a local development URL.

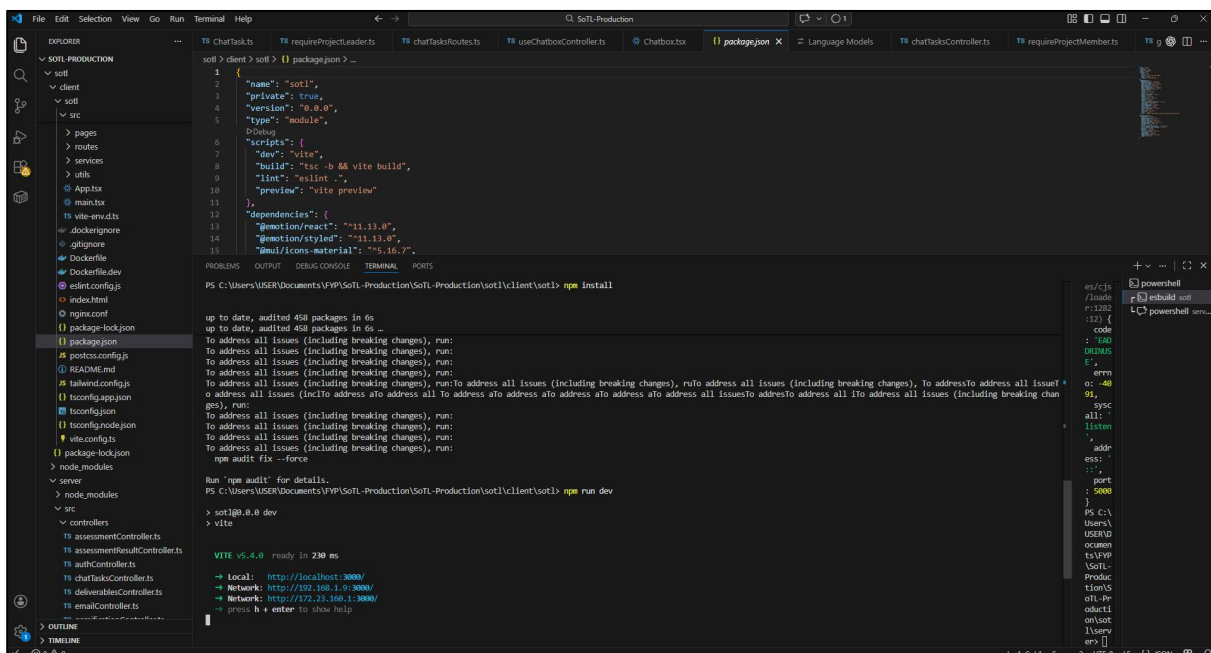


Figure 4.2: React application running successfully with a local development URL

Once the application has started, the system interface can be accessed in the local browser. In this project, React is used to create the personalised chatbox interface that is integrated into the project website or system. Users in the system can send messages, look up for answers, and interact with the task-related features based on their given roles within the team.

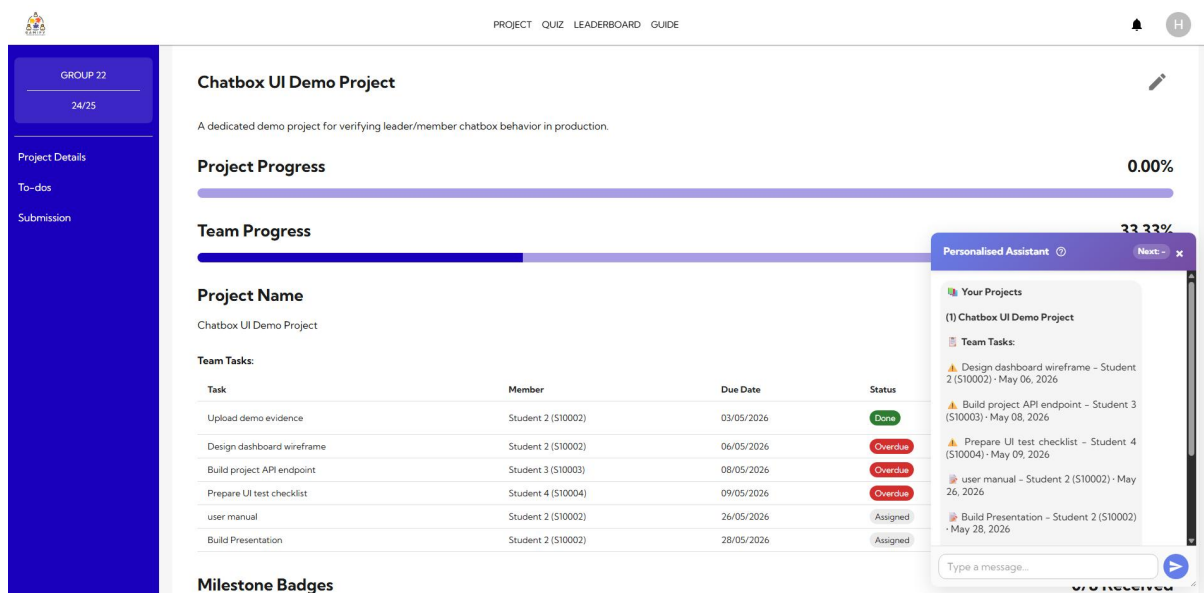


Figure 4.3: Chatbox interface displayed in the system

React is also designed with reusable components to support system organisation and maintainability. The chatbox, message bubbles, input field, and displayed task are treated as discrete modules. This framework makes it easy to upgrade and scale up the system if needed.

In addition, React works with supporting libraries like Material UI and React Markdown. Material UI is used to create the interface using pre-built components, and React Markdown is used to write the chatbot replies within the chat interface. These connections help improve the system's functionality and the chatbox's visual display.

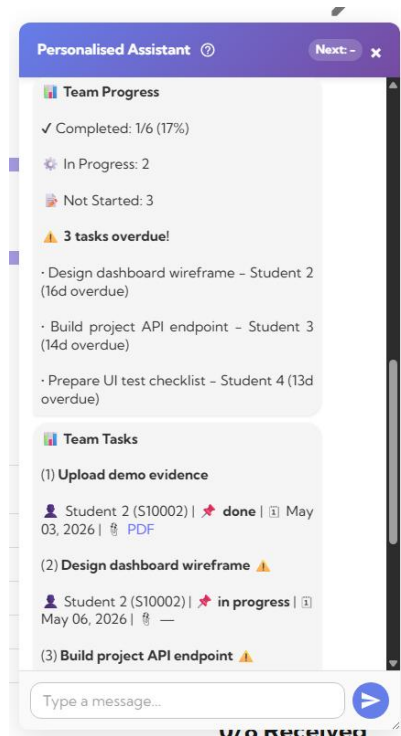


Figure 4.4: Styled chat interface with formatted messages

Overall, React contributes very crucially to the development of the proposed system by supporting an interactive and structured user interface for the proposed personalised chatbox system.

4.2.2 Node.js and Express

Node.js is a JavaScript runtime environment for running server-side applications, while Express is a Node.js-based web framework that simplifies building web APIs. Node.js and Express are used in this project to create the personalised chatbox system's backend. They help oversees the receiving API calls, processing chat commands, managing tasks, and connecting with the database.

To begin, the backend system's runtime environment requires Node.js to be installed. After that, the necessary backend dependencies are installed using the following command: `npm install`

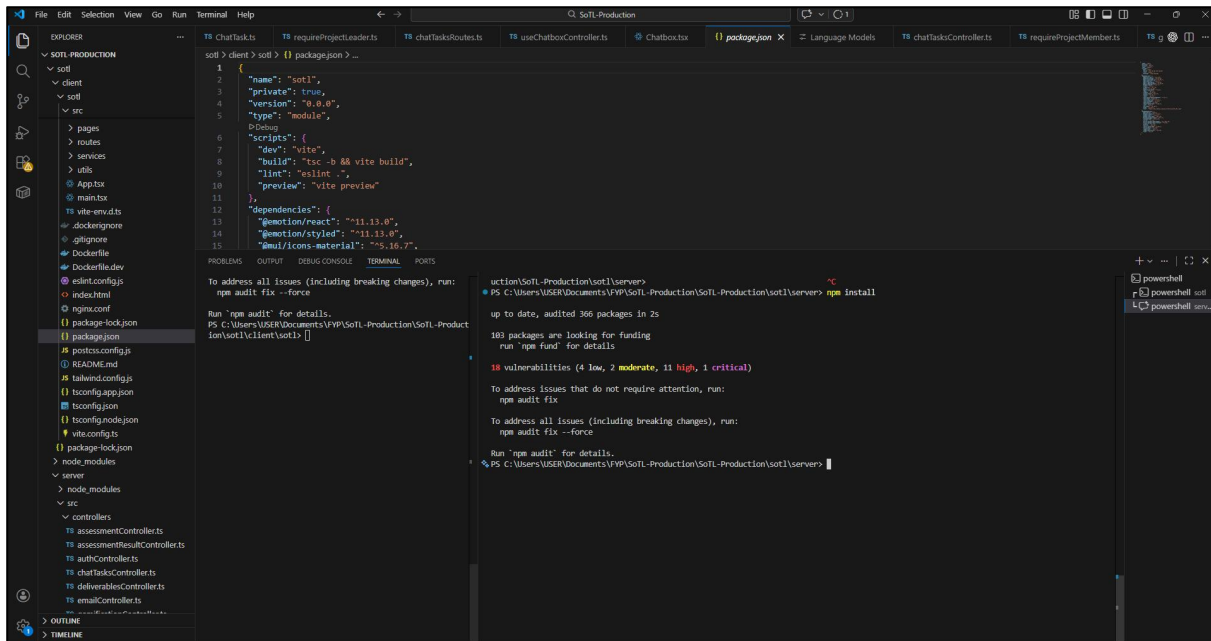


Figure 4.5: Installation of backend dependencies in the terminal

After the installation is finished, the backend server may be launched using the following command: `npm run dev`.

This command starts the server and lets it listen for incoming requests from the frontend. Once the server is up and running, the terminal will show the server port and connection status.

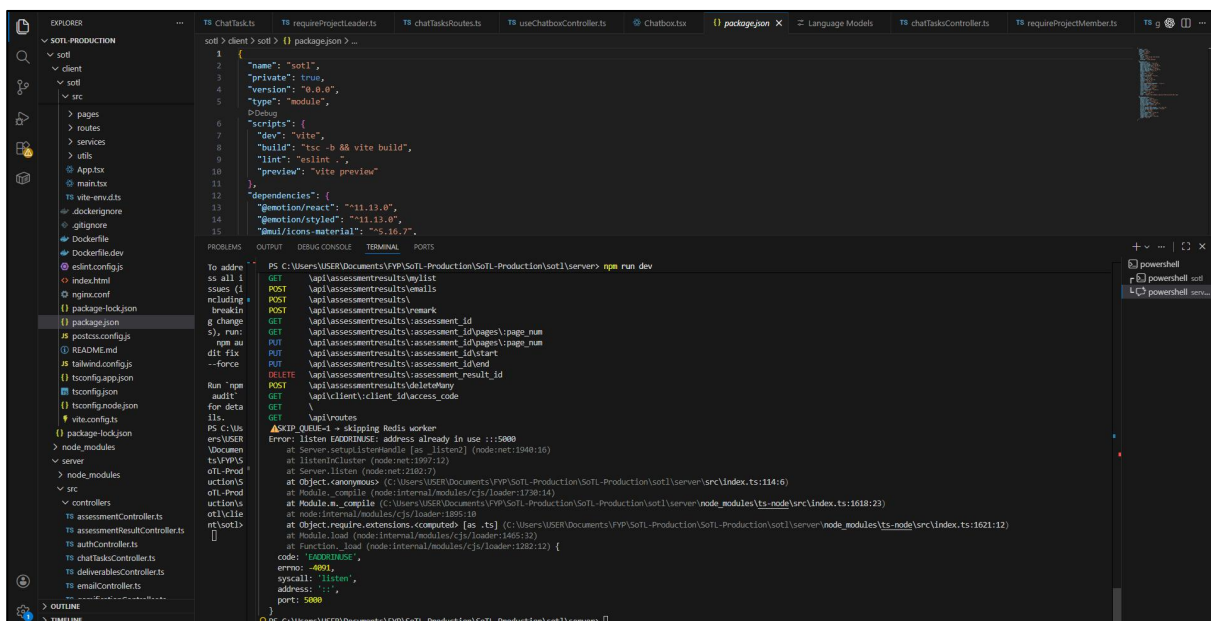


Figure 4.6: Backend server running successfully

The following code snippet shows how the backend API endpoint receives messages from the frontend and processes the request before returning a response.

```
/** POST /api/llm/chat body: { messages: LLMMessage[], rolePrompt?: string } */
router.post("/chat", async (req, res) => {
  try {
    // Debug helper: ?echo=1 just returns the last user message
    if (req.query.echo === "1") {
      const { messages } = req.body as { messages: LLMMessage[] };
      const last = (messages || []).filter(m => m?.sender === "user").pop();
      return res.json({ reply: `[echo] ${last?.text || "(no text)"}` });
    }

    if (!OPENAI_KEY) {
      return res.status(503).json({ error: "Server missing OPENAI_API_KEY" });
    }

    const { messages, rolePrompt } = req.body as {
      messages?: LLMMessage[];
      rolePrompt?: string;
    };
  }
});
```

Figure 4.7: Backend API endpoint receiving chat request from frontend

```
const reply = await generateChatCompletion(openAIMessages.map(m => ({ ...m })));
if (!reply) {
  return res.status(502).json({ error: "AI generation failed: empty response" });
}

return res.json({ reply });
} catch (err: any) {
  const msg = err?.message || String(err);
  console.error("❌ OpenAI error:", msg);
  const dev = process.env.NODE_ENV !== "production";
  return res.status(502).json({ error: dev ? `AI generation failed: ${msg}` : "AI generation failed" });
}
});
```

Figure 4.8: Backend processing request and returning response using ChatGPT API

The backend API implementation is shown in Figure 4.7 and Figure 4.8, illustrating how user messages are received, processed, and returned as responses.

The backend further differentiates between system commands and general user queries. Command-based inputs trigger database operations such as task assignment and updates, while non-command inputs are forwarded to the ChatGPT API for response generation.

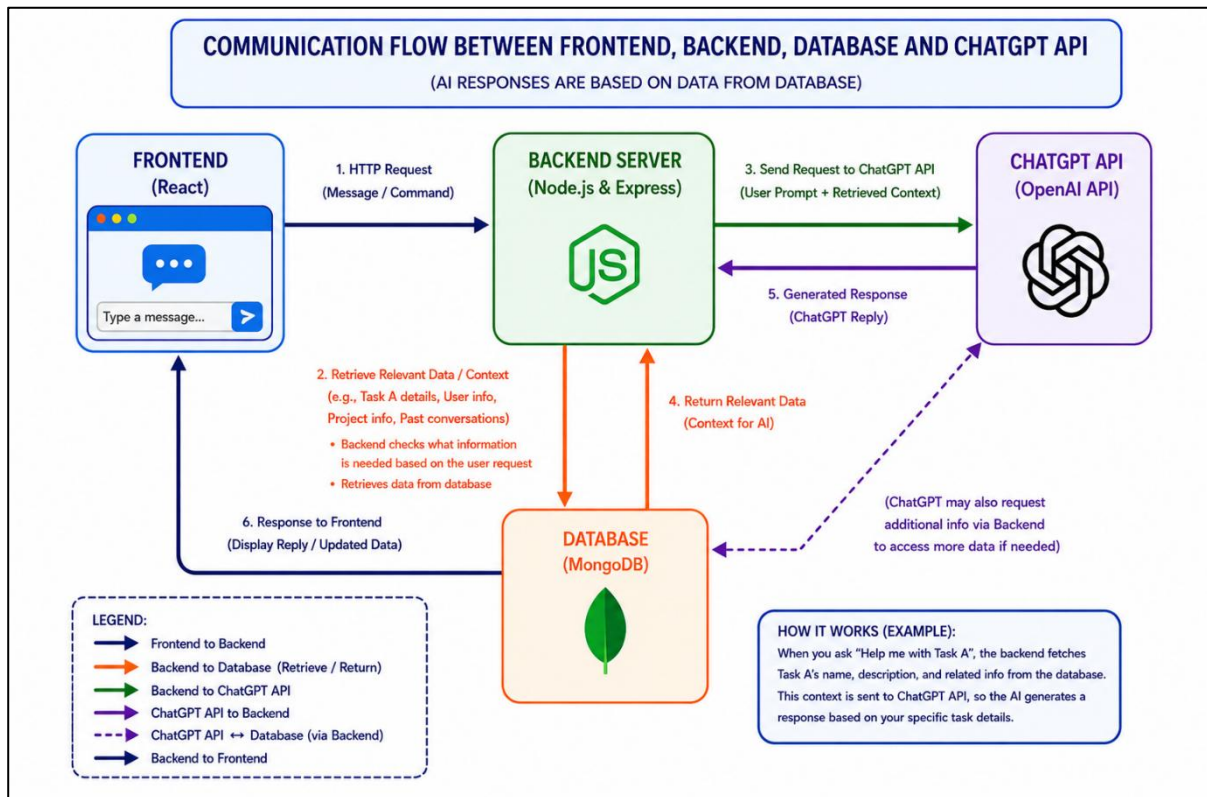


Figure 4.9: Communication flow between the frontend, backend, database and ChatGPT API

The backend server serves as the communication hub between the frontend, MongoDB database, and ChatGPT API, as shown in Figure 4.9. When a user enters a message in the chatbox, the backend looks at the message and decides if it was a request for more information related to a project. When the user request is linked to a set of assigned tasks, project information, deadlines, or user roles, the backend retrieves the data from the database (MongoDB) before passing the request on to the ChatGPT API. Any data that is retrieved, including the task description, project details, user responsibilities, and task progress records, is provided as part of the API request. This allows the ChatGPT API to produce answers that are relevant to the specific project context and tasks. For instance, if a member asks for help in a task, the AI can create instructions based on the task description given by the group leader. This communication mechanism is context-sensitive, which enables the personalised assistant to give more accurate and relevant responses as well as assist in completing tasks and holding groups accountable.

Express is also configurable with middleware in order to manage authentication, role-based access control, and request validation. This ensures that only the allowed users with the right role can carry out certain operations, like assigning tasks or tracking task progress.

Overall, Node.js and Express are crucial to handling the system's backend functionality, which allows for easy communication between the frontend, database, and external AI services.

4.2.3 MongoDB

MongoDB is a NoSQL database management system designed to store and manage application data. It saves data in the form of collections and documents, which allows for the management of both dynamic and organised information. In this project, MongoDB serves as the primary database for the proposed personalised chatbox system. It is in charge of maintaining user information, project details, deliverables, and chat-related task data utilised by the system.

Before connecting to the backend application, MongoDB must first be installed or accessible via a specified database server. After that, the database connection is created in the backend settings, which allows the system to be in touch with MongoDB during execution phase. After the connection is established correctly, the backend may store, retrieve, and update data based on how the user interact with the system.

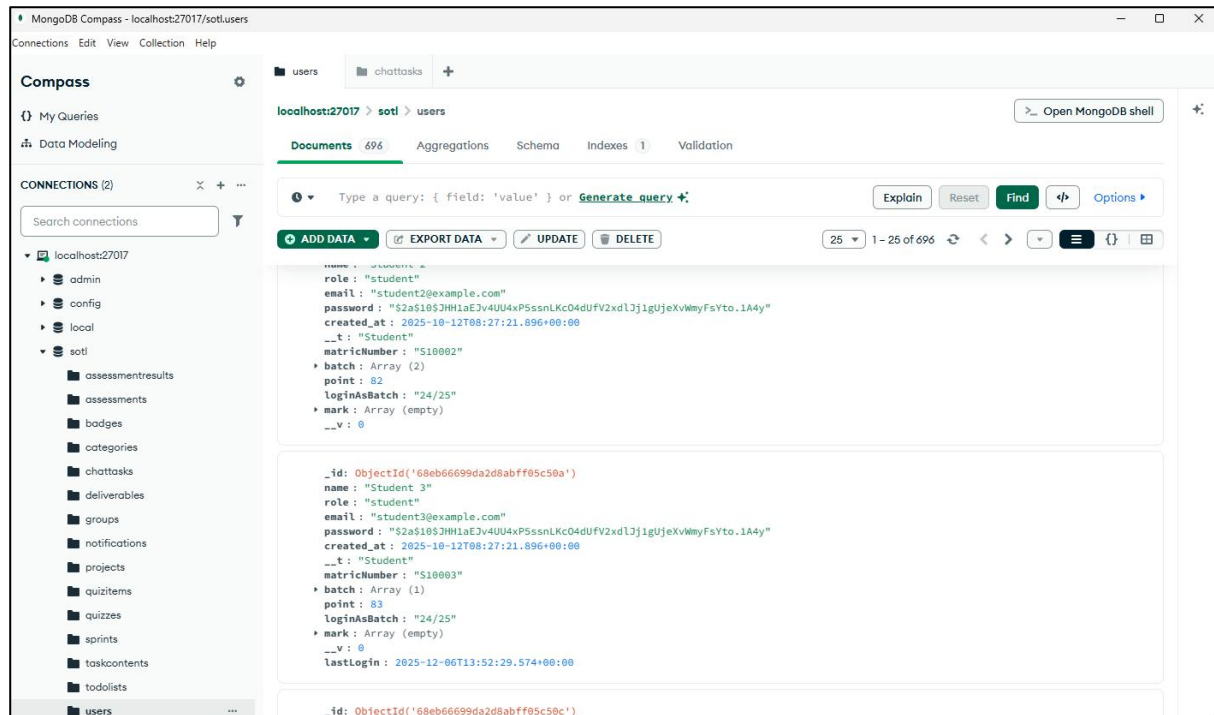


Figure 4.10: Successful connection between the backend server and MongoDB

In this project, MongoDB is set up to provide various critical collections necessary for system functioning. These contain collections for user accounts, project data, deliverables, and chat tasks. The chat tasks collection is very essential since it maintains task assignment details such as the task title, description, assigned member, task status, and evidence link. This enables the system to handle task-related interactions straight from the chatbox.

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
assessmentresults	-	1.36 MB	3.1K	2.63 kB	1	114.69 kB
assessments	-	36.86 kB	6	630.00 B	1	36.86 kB
badges	-	36.86 kB	16	222.00 B	1	36.86 kB
categories	-	40.96 kB	6	3.85 kB	1	36.86 kB
chattasks	-	36.86 kB	5	337.00 B	1	36.86 kB
deliverables	-	36.86 kB	28	123.00 B	1	36.86 kB
groups	-	61.44 kB	164	1.10 kB	1	36.86 kB
notifications	-	73.73 kB	1K	175.00 B	1	57.34 kB
projects	-	163.84 kB	88	2.81 kB	1	36.86 kB
quizitems	-	46.06 kB	54	574.00 B	1	36.86 kB
quizzes	-	36.86 kB	6	777.00 B	1	36.86 kB
sprints	-	36.86 kB	110	89.00 B	1	36.86 kB
taskcontents	-	151.56 kB	3K	265.00 B	1	110.59 kB
todolists	-	57.34 kB	352	171.00 B	1	46.06 kB
users	-	86.02 kB	696	349.00 B	1	49.15 kB

Figure 4.11: Collections created in MongoDB for the proposed system

MongoDB is also used to do a data retrieval and update the activities depending on user responsibilities. Leaders, for example, can provide tasks to members, who can then update their progress and offer evidence as well in the system. All these activities are mirrored in the database via insert, update, and query operations performed by the backend. This guarantees that the information shown in the chatbox matches with the real task records that are maintained in the system.

Furthermore, MongoDB is designed to handle changing task structures and project-related data, making it ideal for the customised chatbox system. MongoDB enables for effective management of many sorts of data in the system, including user role information, task progress, deadlines, and conversational recordings, without the need for a traditional table structure.

Overall, MongoDB is critical to the proposed system since it serves as the fundamental data storage component that enables task management, project monitoring, and role-based interactions within the chatbox.

4.2.4 ChatGPT API

The ChatGPT API is an API that permits the system that would send the LLM prompts and receive intelligent and context-aware responses from the LLM. In this project's backend, we are using the ChatGPT API to offer customisable chatbox. Through the chat interface, the ITA solution allows the system to provide role-based coaching, ideas for tasks, and conversational answers according to user input.

At first, we set up our ChatGPT API by collection our API key and placing it safely in our backend env file. This key is used to authenticate requests exchanged between the back-end server and the AI service. When activated, the backend can connect with ChatGPT API and manage the user requests dynamically.

When a user sends a message through chatbox UI, the React front-end takes the input and sends the same to the backend server through an API. The backend checks the message and determines whether it has a particular system command and task assignment or task update. If

the message is not a command, it is sent to the ChatGPT API with the necessary context, such as the user's role and job.

Following a request, the ChatGPT API generates a relevant response based on the information and context provided. The answer prepared is then sent to the backend server which forwards it to the front end. The chatbox interface indicates the response so that users may obtain fast feedback and instruction.

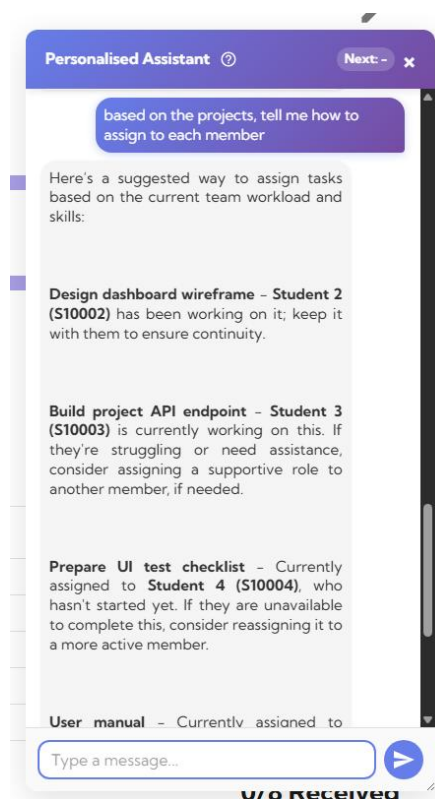


Figure 4.12: Example of a response generated by the ChatGPT API in the chatbox interface

Integration of ChatGPT API enhances system functionality by adding smart discussions and personalised assistance capabilities. For instance, leaders can receive guidance from their superiors on handling team activities and monitoring progress, while members can ask for assistance in carrying out assigned tasks. It boosts cooperation, helps in making decisions, and reduces chances of misunderstanding in group activities.

Overall, the ChatGPT API is essential to provide real-time engagement, role-based advice, and intelligent support within the personalised chatbox system.

4.2.5 Docker and Nginx

Docker is a containerization technology that enables developers to package and run applications in isolated environments called containers. In this project, we will use Docker to deploy the frontend, backend and database services consistently across all environments. It ensures that the system can operate reliably in multiple environments with no dependencies.

Initially, the installation of Docker and Docker Compose takes place to facilitate containerisation. Docker Compose refers to and manages the various services of the system, including React frontend, Node.js backend, MongoDB database and Nginx server.

These services are constructed using docker-compose. The yml file explains how each component is created, connected, and executed.

After the configuration, we may run all system components at once using the command: docker-compose up.

This command will create and launch all the containers such that the front end, back end and database can communicate with one another.

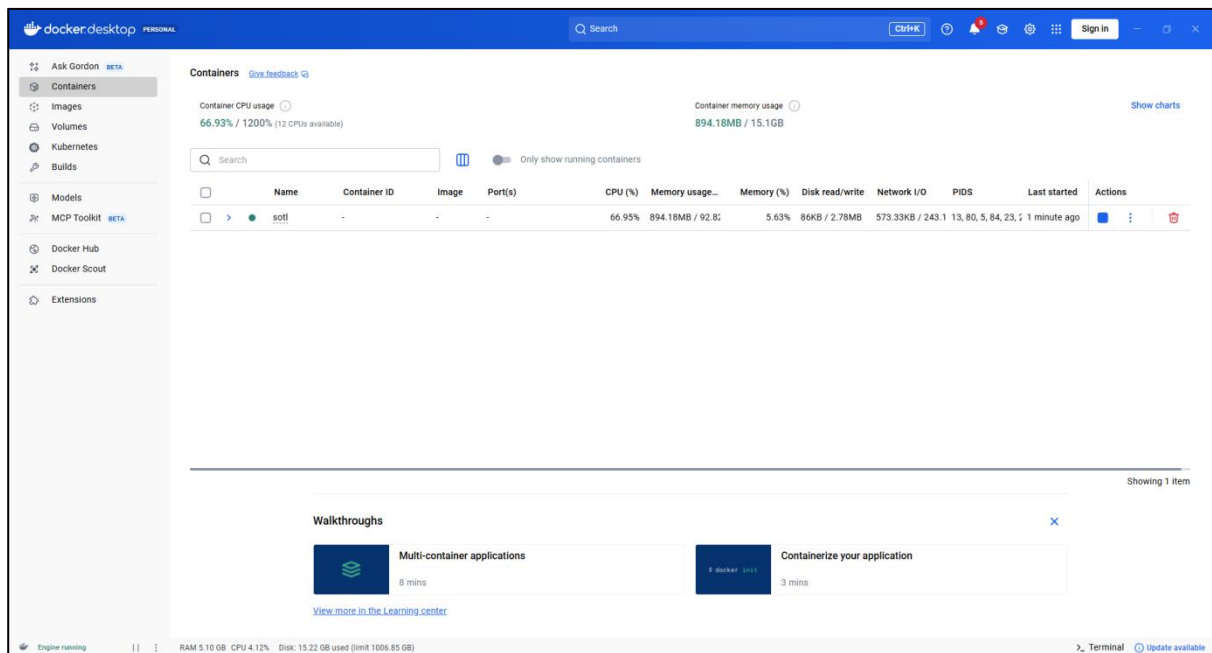


Figure 4.13: Docker containers running successfully

In this project, Nginx serves as a reverse proxy server, which directs all the incoming requests to the relevant backend services. It ensures that API calls from the frontend are correctly routed to the backend server, which helps improving overall system speed and scalability.

Docker helps simplifies the deployment process by summarising all dependencies and configurations into containers. This helps eases up setup and assures that the system will always performs consistently during development and deployment phase.

Overall, Docker and Nginx play a very critical roles in this proposed system, offering a dependable and scalable environment for executing and managing all system components.

4.3 Role Based Access

Role-Based Access is a technique for controlling system functionality based on the responsibilities allocated to users. The proposed personalised chatbox system uses role-based access control to separate users' duties and interactions, especially between the Leader and Member roles. This guarantees that each user may only access features that are relevant to their role in the group.

When a person joins a project group, the system assigns them a role. The role controls the types of activities the user may perform in the chatbox and task management system. The Leader manages the group's overall progress, while Members do specified tasks.

The system provides support for the Leader position by allowing the leader to give tasks to the team members, track task completion, and check the overall project status. The Leader can send task-related directives via the chatbox interface, such as assigning assignments, establishing deadlines, and reviewing completed work. The backend processes these actions and stores them in the database to ensure that group activities are properly tracked.

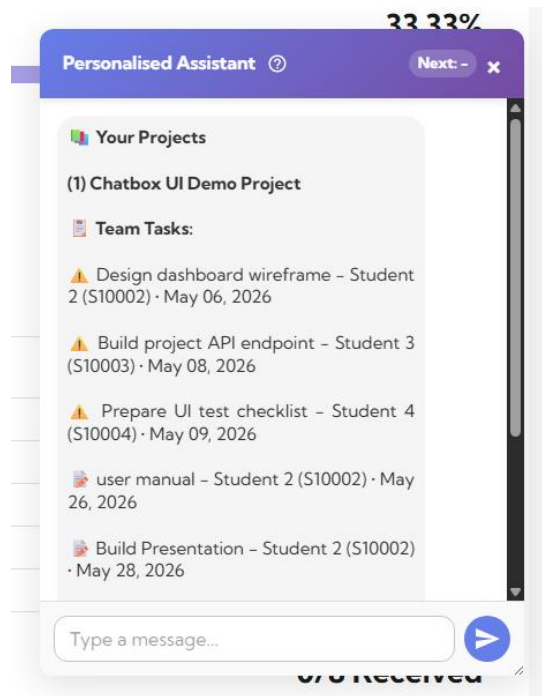


Figure 4.14: Leader interface in the chatbox system

The system includes functions for the Member role, such as accessing the assigned tasks, updating task progress, and reporting evidence of job completion. Members use the chatbox to get reminders, check deadlines, and update their task statuses. This guarantees that each member is held accountable for their allocated obligations.

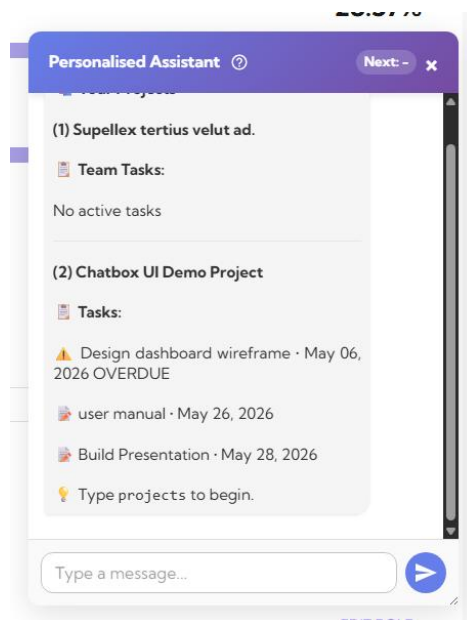


Figure 4.15: Member interface in the chatbox system

The role-based access method is enforced at the backend level by middleware and role verification logic. When a request is made, the system determines the user's role before granting access to specified features. For example, only Leaders may assign tasks, whereas Members can only update their own. This prohibits unauthorised activity and ensures system integrity.

In addition, the chatbox offers tailored advice based on the user's position. Leaders receive advice for work management and team collaboration, whilst Members receive assistance on task completion and progress updates. This improves the system's usability and guarantees that users are supported based on their responsibilities.

Overall, implementing role-based access in the proposed system improves work coordination, increases responsibility, and decreases the issue of free riding in group assignments by explicitly defining user roles and system interactions.

4.4 Workflow for System's Application

The system workflow describes the interaction of the users with the chatbox system which is personalised and the response of the system to this input. It explains how data moves from the frontend to the backend to the database to the ChatGPT API to handle tasks and roles.

Initially, the user logs into the system and selects a project. When the project is selected, the chatbox appears, showing relevant information about the selected project, including upcoming deadlines, task descriptions, and reminders for members. The information display in the chatbox varies per user's role in the team, with the Leaders getting project-level updates and the Members getting task-level information.

When a user sends a message from the chatbox, it happens on the frontend, then it goes to the backend. The backend evaluates the message to check whether it is a specified command or a generic query. For example, the system communicates instructions, such as task assignments and updates, directly to support task completion.

If it is detected that a message is a command, the backend will take the required action, like changing task data in the database or assigning a task to a particular member. The updated information is sent back to the front end and displayed in the system's chatbox.

However, if the message is detected as a generic question, it will be sent out to the ChatGPT API. After processing the given input, the API will create an appropriate response depending on the context provided, which includes all the details about the role of the user and the task given to them. The user will see all the responses given by ChatGPT in the chatbox after it has been returned to the backend system.

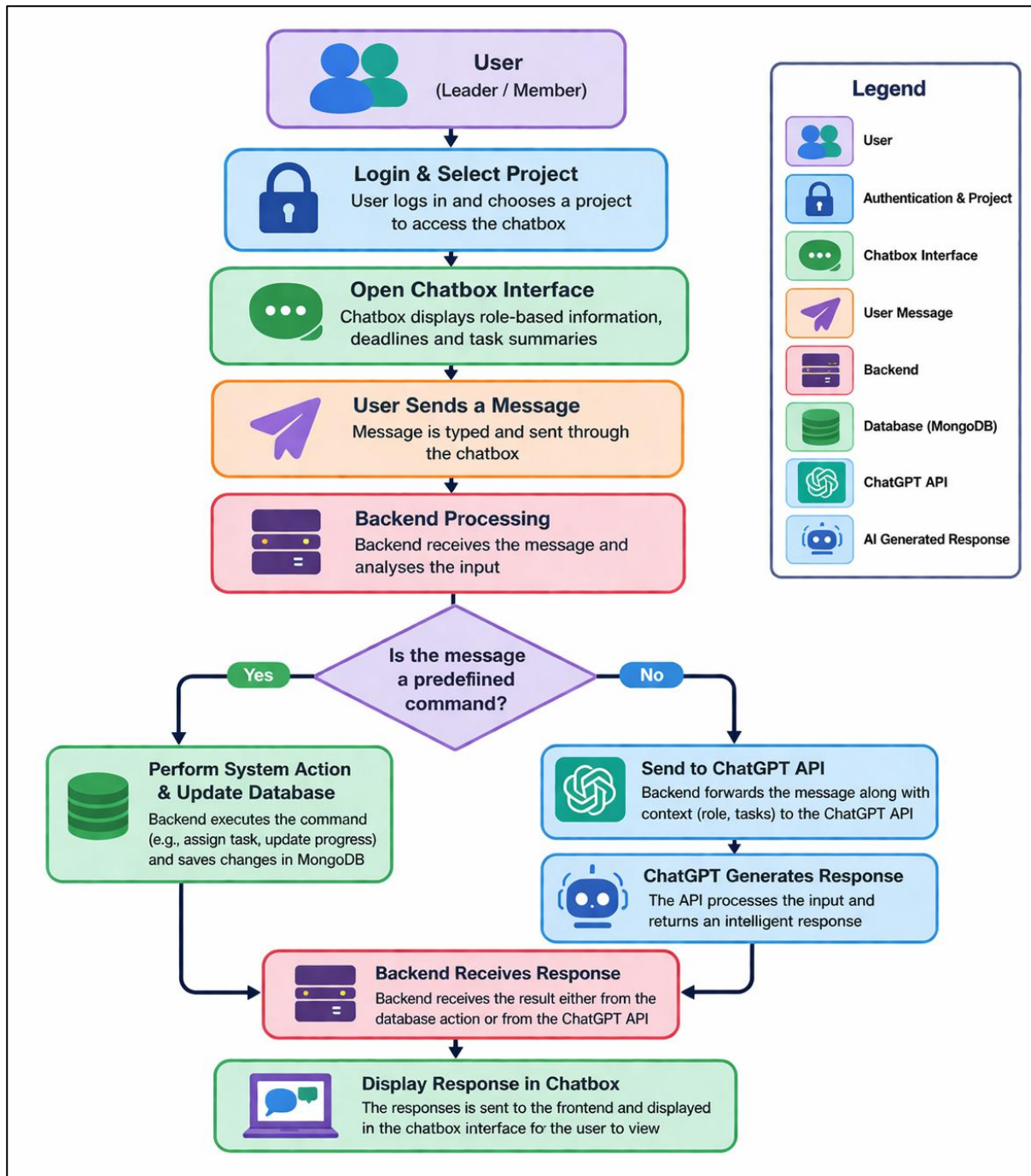


Figure 4.16: Overall workflow of the personalised chatbox system

This workflow ensures that system-driven actions and AI-generated answers are efficiently managed inside a single interface. It also allows all the users to handle tasks and get instructions without leaving the chat space.

In general, this system workflow helps fuse multiple components to give a mixed user interaction, which allows for real-time engagement, organised task management, and personalised help within group projects.

4.5 Modules for System's Application

The proposed personalised chatbox system is to be developed with the help of various functional modules where each module is to be used for specific functioning. The modules help AI to chat, coordinate tasks, generate intelligent replies and team-oriented guidance. The system implementation will become more systematic, maintainable, and in accordance with the objectives of the system discussed in previous chapters by modularising the system.

4.5.1 Chatbox Interaction Module

The user and the system are interacting through the Chatbox Interaction Module which acts as mediator. Chatbot's real-time communication offers timely responses when users send messages successfully through a personalised chatbox. The messages will be dynamically rendered using graphics React that will provide smooth interaction.

The chat window captures what the user types and sends it to the backend server. The answers that the system generates are then shown back in the chatbox for further interaction. This module is a necessary module that is our primary means of communication for the entire system.

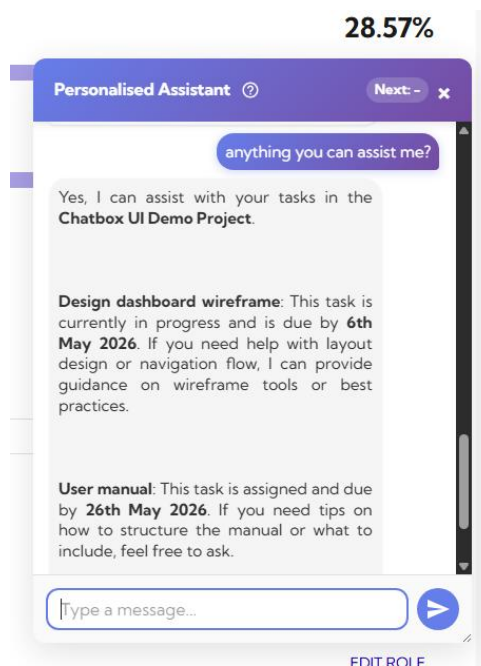


Figure 4.17: Chatbox Interaction Interface

4.5.2 Task Management Module

The task management module is for the task-related operation of the system. This software allows Leaders to assign group tasks, set deadlines and monitor task completion. Members

can view tasks assigned to them, update the status of assigned tasks and submit evidence of completion.

The backend is responsible for processing all task-related data, which is then stored in MongoDB. This means that the information regarding the task is updated and reflected in the system. Effectively tracking individual contributions helps improve coordination among the group members while supporting accountability.

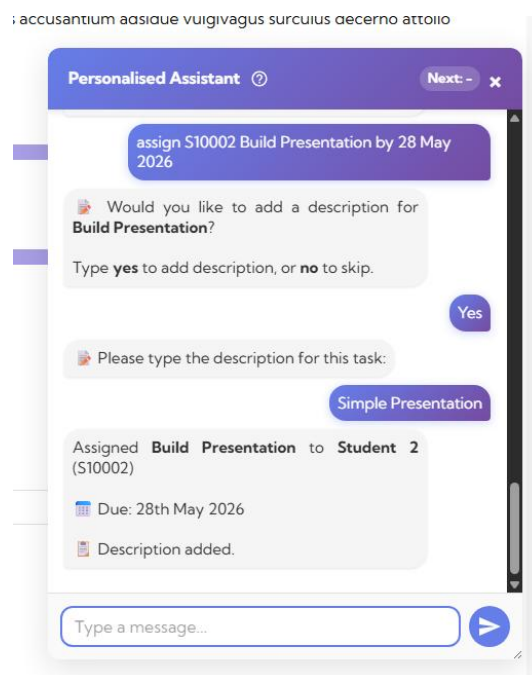


Figure 4.18: Task Management Functionality

4.5.3 AI Response Module

The AI Response Module provides intelligent, context-aware replies to user queries. When the user sends some general messages, then based on a particular situation, the system will generate an appropriate reply based on the user's role.

The AI Response Module also leverages project-specific data pulled from MongoDB, prior to generating responses. The backend retrieves information from the database given a user's questions and adds it to the request sent to the ChatGPT API as contextual information that relates to the user's assigned tasks, project deadlines, or group responsibilities. This allows the AI to provide suggestions given specific descriptions of the tasks, project needs, and the

user's part in the group. Therefore, the answers are not generic answers of a chatbot but personalised for the user's current project context.

This module allows users to receive guidance, suggestions, and explanations directly through the chatbox. In general, it improves overall interaction by making the system more responsive and supportive, helping users carry out their tasks.

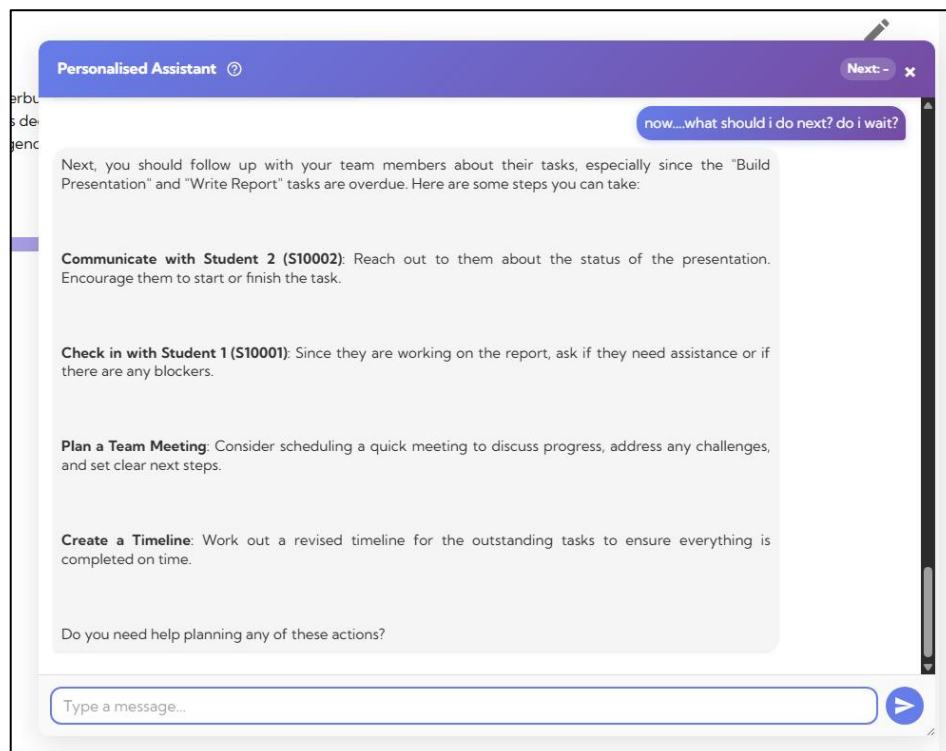


Figure 4.19: AI-Generated Response in Chatbox

4.5.4 Reminder and Notification Module

Alerts related to tasks and deadlines are provided by the Reminder Notification module. Users will see these reminders in the chatbox so they are aware of pending items and upcoming deadlines.

This module will reduce non-delivery and prompt the user to stay connected with their assignment. Notifications appear neatly within the chat, making them easy to notice.

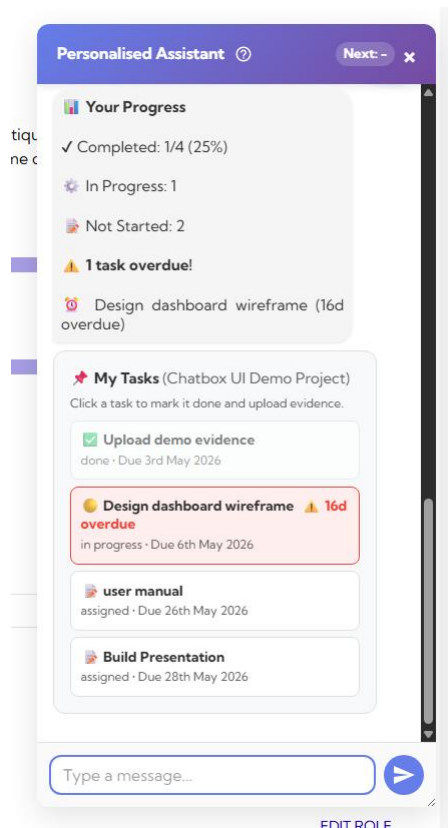


Figure 4.20: Reminder and Notification Display

4.5.5 Database Management Module

The Database Management Module allows for data storage and organization of systems. Having the capacity of maintaining information like tasks, user roles, chats, etc when a user interacts with the system.

The module will provide a way to capture any data the system needs at the right time and ensures that all changes are made accurately. It is essential for ensuring uniformity and assisting the system's essential functioning of the system.

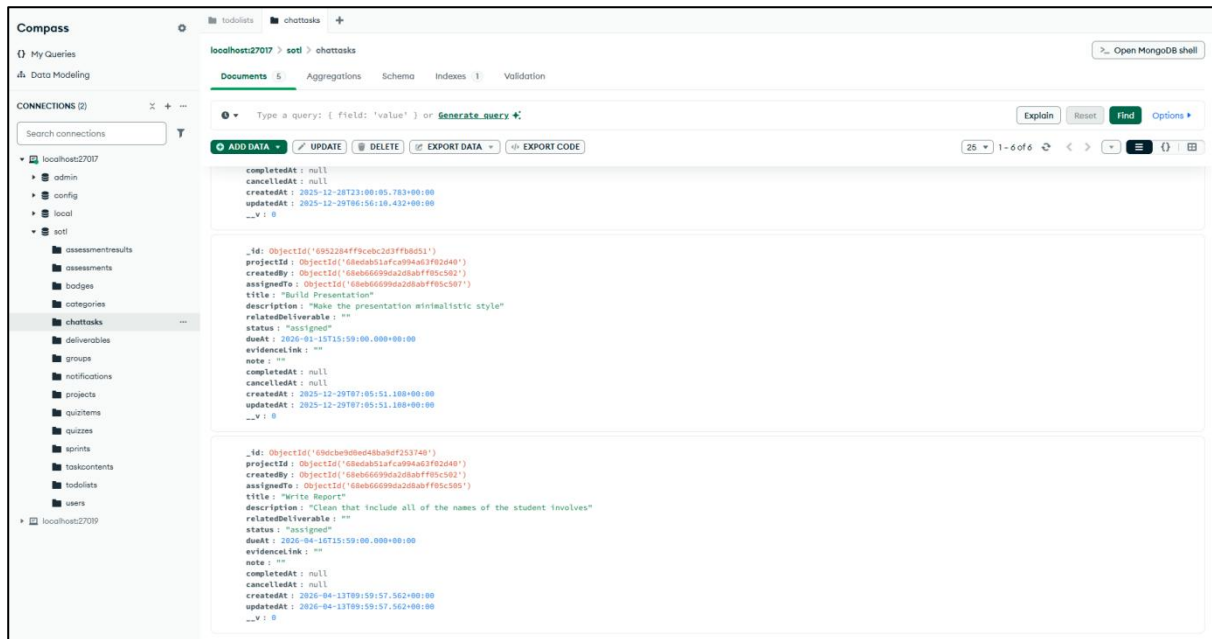


Figure 4.21: Database Structure for the Chatbox System

4.6 Summary

The chapter presented a personal chatbox for role-based guidance as well as group accountability as per the proposal. The process involves installing and configuring components and integrating the frontend, backend, database, and AI services.

This implementation also shows the applicability of role-based access to differentiate the functional aspect between Leader and Member, and allows the user to use the system respectively. Also, the system workflow indicates how user input is parsed as an order for doing tasks or as general questions and the generation and display of suitable answers through the chatbox.

The system is modular which has different functionalities such as chatbox, task-related activities, AI response, reminder and notification, database management etc. By combining tools for live communication with mods for supporting tasks and presenting users with personalised guidance.

In conclusion, the personalised chatbox system successfully enables conversational interaction with task management features, thereby improving group coordination, enhancing accountability, and reducing free-riding issues among students in group projects.

CHAPTER 5: TESTING

5.1 Overview

The verification of the personalised chatbox system for role-based guidance and group accountability is ensured in this chapter fulfilling the user requirements and the system requirements as well. Moreover, testing is vital in ensuring that the proposed system is functioning properly without any critical bugs and is ready for use in the education world. The system will be tested at every stage of the development process to ensure that all its features are functioning properly. This includes the chat and interaction feature, task management, role-based guidance, and the reminder system. Continuous testing assists in detecting and fixing flaws early on, thereby enhancing overall reliability of the system. The chapter testing is further divided into functional testing and non-functional testing and record all test results from each conducted test to further support system evaluation and improvement of any future enhancement of proposed personalised chatbox system.

5.2 Functional Testing

Functional testing to check whether all features and functionality of the system work well. This consists of testing the compatibility of the frontend and backend, along with the database and ChatGPT API. Also ensuring all role-based features work fine and all operation of task manager work well.

5.2.1 System Testing

System testing evaluates the entire system and all of its aspects as a complete package to ensure coherence and integration. The simulated actual users chatbox system that was used by a leader and members for our proposed system.

The following key functionalities are being tested:

- Chat message input and response generation
- Task assignment by Leader to the Members
- Task progress updates and completion
- Role-based access control
- Project selection and task retrieval
- AI-generated responses via ChatGPT API

Table 5.1: System Testing Results

Test Case	Description	Expected Result	Actual Result	Status
Chat Interaction	User sends message	System returns response	Response displayed correctly	Pass
Task Assignment	Leader assigns task	Task appears in member list	Task successfully assigned	Pass
Task Progress Update	Member updates status	Status updated in system	Status updated correctly	Pass
Task Completion	Member marks task as done	Task marked as completed	Task completed successfully	Pass
Role Access Control	Member tries leader action	Access denied	Access restricted correctly	Pass
AI Response	User asks question	AI generates response	Response displayed correctly	Pass

These results shows that the system functionality are working properly without any error that needed to be fixed

5.2.2 Unit Testing

Unit testing focuses on evaluating and testing all the individual features of the system to ensure all the listed module function perfectly on its own.

The following components that are being tested:

- Backend API endpoints
- Chat processing logic
- Role validation functions
- Database operations (insert, update and retrieve)

Table 5.2: Unit Testing Results

Component	Function Tested	Expected Result	Status
API Endpoint	Create task	Task stored in database	Pass
API Endpoint	Retrieve tasks	Correct task list returned	Pass
Role Validation	Leader access	Access granted	Pass
Role Validation	Member restriction	Access denied for restricted actions	Pass
Database	Update task status	Status updated correctly	Pass

These results shows that the individual components are working properly and provide support to the system itself.

5.3 Non-Functional Testing

Non-functional testing – used to evaluate the non-functional aspect of the software systems. In this project usability testing will be done to assess how easily a user can use the personalised chatbox system.

5.3.1 Usability testing

An evaluation was done on the ease of use, clear interactions and other facets to determine user satisfaction of the system. A small group of students were engaged in testing the chatbox features and gave feedback after using it.

5.3.1.1 System Functionality of the Proposed System

This segment assesses how well users can interact with the system to achieve their goal.

User managed to:

- Send messages and receive instant responses
- Use chat commands to view the given tasks and update their progress
- Understand the task assignment and the deadline clearly
- Interact with the role-based guidance which are provided by the system

The outcome indicates that the system effectively facilitates user interaction via a conversation interface. By featuring task management in the chatbox, users can take actions without leaving the space and can improve efficiency and convenience.

Role-based responses further allow users to know their role. Leaders are reminded how to manage the task and members are reminded and instructed on them. It enhances cohesiveness and lessens misunderstanding in group work.

5.3.1.2 User Interface Design of the Proposed System

This section examines the visual design and usability of the chatbox.

The chatbox is designed to be:

- Easy to understand

- It must be easy to understand visually with appropriate placement of message.
- Responsive and interactive
- Consistent with modern chat applications

Key interface features include:

- Left and right message alignment for readability
- Clear input field and send button
- Display of reminders and task information within the chat
- Clean layout with minimal complexity

Users say the interface was simple to use and little learning was needed. The layout of the messages allows for easy interaction, while the overall structure makes it easier to read.

Overall, the interface design enables users to communicate effectively, thus enhancing the user experience with the system.

5.4 Summary

The proposed personalised chatbox system was tested and evaluated in this chapter. Functional testing indicates that all system features, chat interactions, task management, and role-based access are functioning properly and meeting system requirements.

Usability testing was performed to check the non-functionality of the system. This shows that the system is easy to use. Also, it is efficient and can help the group coordinate and be accountable for their activity. The interface enables easy conversation while the role-based guidance enhances better clarity and effective communication amongst the group members.

Overall, the proposed system can achieve the desired results. Which provides an interactive, reliable, and user centred system. That aims at improving teamwork, accountability and communication.

CHAPTER 6: CONCLUSION AND FUTURE WORKS

6.1 Overview

This chapter wraps up the work on the proposed chatbox system for role-based guidance and group accountability. The project goals were achieved, system development was hampered by some issues, and some recommendations are made for scope of future work. By developing, implementing, and testing the system as outlined in the previous chapters, the system is evaluated to see if communication, task coordination, and accountability within student group projects in an educational setting are supported.

6.2 Objective Achievement

Table 6.1: Objective Achievement

Objective	Implementation / Achievement	Status
To analyse existing educational chat systems and their personalisation methods.	Existing systems such as Trello, Google Classroom, and Jira were analysed in Chapter 2 to identify their strengths, weaknesses, and limitations in supporting role-based guidance and group accountability.	Achieved
To design and develop a chatbox that provides role-based communication and guidance.	A personalised chatbox system was successfully designed and developed using React, Node.js, Express, MongoDB, Docker, and ChatGPT API. The system supports role-based communication, task management, reminders, and AI-generated responses.	Achieved
To test and evaluate the system's usability and functionality.	Functional testing and usability testing were conducted in Chapter 5. The results showed that the system operated correctly, supported user interaction effectively, and improved communication and accountability among group members.	Achieved

6.3 Project Limitations

Nonetheless, the development of the proposed personalised chatbox system was successfully done, but there are limitations in this system.

First, the proposed system does not propose an advanced functionality for the lecturer or administrator side. The only interaction that the system supports is between the group leaders and the group members for group projects.

The ChatGPT API and internet connection affect the AI-generated responses. Due to unstable internet connection or API limitations, the responsiveness and performance of the chatbox system may be affected.

Furthermore, the proposed system is currently limited to basic task management functionalities like task assignment, progress updates, and reminders and evidence submission. At the present time, more sophisticated project-management features such as meeting-scheduling or real-time group analytics are out of scope.

Moreover, only a limited user testing and evaluation took place due to time and project constraints. Consequently, an overall evaluation of the system may not fairly represent all users and learning environments.

Even with these limitations, the proposed system could meet its primary objectives and demonstrate how personalised conversational systems can enhance group accountability and coordination in educational settings.

6.4 Conclusion

To sum up, the personalised chatbox system for role-based guidance and group accountability was successfully designed, developed, implemented, and tested in an educational website. The system utilizes a chatbox for conversational interaction, task notifications, reminders, and role-based instructions for group coordination and accountability of the students.

The Rapid Prototyping Model was used for this proposed project. This model gives us a lot of scope for improving the system continuously. The proposed system was implemented with the help of React, Node.js, Express, MongoDB, Docker and ChatGPT API.

The proposed system can run correctly and support system requirements from the results of the test and evaluation. The chatbox accomplished personalised assistance, task awareness, and role-sensitive in-group guidance in assignment to users, which made the communication effective and lessened free-riding in groups.

Overall, the project exhibited that the incorporation of AI-powered conversational systems into learning environments enhances collaboration, accountability and engagement of users in student group projects.

6.5 Future Works

Future models of the proposed personalised chatbox system can undergo various enhancements for better performance.

To facilitate group collaboration, uses and features such as meeting scheduling, shared calendars, and group discussion summary chatbox modification are possible.

Furthermore, they may include advanced artificial intelligence capabilities as functionalities like sentiment analysis, smarter contextual understanding, and adaptive learning support in future improvements that can generate more personalised and intelligent responses for users.

Aside from that, the system can also be improved to support the lecturer and administrator functions that would enable the lecturer to see the group progress, monitor participation, and even guide them through the system.

In addition, a mobile app can be developed in the future to allow users to access the chatbox system conveniently through their smartphones and tablets.

Further usability testing on a larger scale with larger respondent pool from various education background and the proposed system may be tested in real education settings in future studies.

REFERENCES

- Alaoui, Y., Belahbib, A., Bouhorma, M., & El Achaak, L. (2020). Unified process to design and develop serious games for schoolchildren. In *Proceedings of the 2020 International Conference on Networking, Information Systems & Security (NISS 2020)* (pp. 1–8). Association for Computing Machinery. <https://doi.org/10.1145/3386723.3387831>
- Bieletzke, S. (2024). SMARTA (Student Motivation and Reflective Training AI-Assistants): Chatbots as individual study coaches for tackling the Two Sigma Problem. *EDULEARN24 Proceedings: 16th International Conference on Education and New Learning Technologies*, 2422–2430. <https://doi.org/10.21125/edulearn.2024.0667>
- Borsci, S., Kurosu, M., Federici, S., & Storni, C. (2023). An empirically based model of software prototyping: a mapping study and a multi-case study. *Empirical Software Engineering*, 28(7), 1–35. <https://doi.org/10.1007/s10664-023-10331-w>
- Elkot, M. A., Alhalangy, A., AbdAlgane, M., & Ali, R. (2025). Pedagogical influence of AI-chatbots on learning outcomes: A systematic review. *International Journal of Educational Methodology*, 11(4), 527–540. <https://doi.org/10.12973/ijem.11.4.527>
- Kim, N. Y., & Lim, J. (2025). AI chatbot design principles to enhance collective efficacy in collaborative learning. *Asia Pacific Education Review*. <https://doi.org/10.1007/s12564-025-10080-z>
- Labadze, L., Grigolia, M., & Machaidze, L. (2023). Role of AI chatbots in education: Systematic literature review. *International Journal of Educational Technology in Higher Education*, 20, 56. <https://doi.org/10.1186/s41239-023-00426-1>
- Lin, M. P.-C., & Chang, D. (2023). CHAT-ACTS: A pedagogical framework for personalized chatbot to enhance active learning and self-regulated learning. *Computers and Education: Artificial Intelligence*, 5, 100167. <https://doi.org/10.1016/j.caeai.2023.100167>
- Molla, M. M. I., et al. (2024). A comparison of transforming the user stories and functional requirements into UML use case diagram. *International Journal of Innovative Computing*, 14(1), 29–36. <https://doi.org/10.11113/ijic.v14n1.463>
- Nurfauza Jali, Cheah Wai Shiang, Suriati Khartini Jali, Syahrul Nizam Junaini and Tan Ping Ping. 2026. Playing the Project: Elevating Software Development Team Performance through Gamification. In 2026 15th International Conference on Software and

Computer Applications (ICSCA 2026), February 03-05, 2026, Langkawi, Malaysia.
ACM, New York, NY, USA, 18 Pages.

Stöhr, C., Ou, A. W., & Malmström, H. (2024). Perceptions and usage of AI chatbots among students in higher education across genders, academic levels and fields of study. *Computers and Education: Artificial Intelligence*, 7, 100258.
<https://doi.org/10.1016/j.caeai.2024.100259>

Universiti Malaysia Perlis. (n.d.). *UniMAP eLearning platform*.
<https://elearning.unimap.edu.my/>

APPENDIX A

Survey Questions:

12/30/25, 1:44 PM

Personalised Chatbox for Role-Based Guidance and Group Accountability

Personalised Chatbox for Role-Based Guidance and Group Accountability

Principle Researcher:

I am Lleyton Geboh Anak Leslie (84354), a final year in Multimedia Computing at Faculty Computer Science and Information Technology (FCSIT), Universiti Malaysia Sarawak (UNIMAS). I am currently undertaking my Final Year Project (FYP) and will develop a **Personalised Chatbox for Role-Based Guidance and Group Accountability** in an educational website which is Selab Management System. My FYP supervisor is Ts. Nurfausa Bt. Jali.

Participants Guideline:

- Participation is completely **voluntary**.
- All responses are **anonymous** and kept confidential.
- You may stop responding at any time without penalty.

This survey consists of three sections:

- Section A: Demographic Profile
- Section B: User Habits & Perception
- Section C: Perception Toward an AI-Powered Personalised Chatbox for Group Work & Accountability in Educational Website

* Indicates required question

Participant Consent Form:

You are invited to participate in a research study titled:

“Personalised Chatbox for Role-Based Guidance and Group Accountability”

for Final Year Project (FYP) at Universiti Malaysia Sarawak (UNIMAS).

Before taking part, please read the following carefully to understand the purpose of this study

Purpose of Research:

This research gathers information regarding your user experience, preferences, expectations, and opinions about the personalised chatbox system. Your feedback is highly valued and will contribute significantly to improving the system’s design and effectiveness.

If you volunteer to participate, you will be asked to complete this questionnaire, which will take approximately **5–10 minutes**. Your responses are completely **anonymous** and your identity will not be revealed.

You may stop at any time without penalty or consequences.

1. 1. Consent to Participate:

★

I agree to take part in this study and understand that participation is voluntary, anonymous, and I may withdraw at any time without consequences.

Mark only one oval.

- ☐ Yes, I agree to participate
- ☐ No, I do not agree to participate (End form)

Section A: Demographic Profile

2. 2. Age *

Mark only one oval.

- ☐ 18–24
- ☐ 25–30
- ☐ 31–40
- ☐ 41–50
- ☐ 51 and above

3. 3. Gender *

Mark only one oval.

- ☐ Male
- ☐ Female

4. 4. Academic Status *

Mark only one oval.

- ☐ Undergraduate
- ☐ Postgraduate
- ☐ Working adult
- ☐ Unemployed

5. 5. Have you used AI chatbots before? *

Mark only one oval.

- ☐ Yes
- ☐ No

Section B: User Habits & Perception

6. 6. How often do you use chatbots (ChatGPT, Gemini, etc) *

Mark only one oval.

- ☐ Daily
- ☐ Several times a week
- ☐ Once a week
- ☐ Rarely
- ☐ Never

7. 7. For what purpose do you usually use chatbots? (Tick all that apply) *

Tick all that apply.

- ☐ Academic work
- ☐ Coding/programming
- ☐ Group projects
- ☐ Personal tasks
- ☐ Entertainment
- ☐ Other: _____

8. 8. How often do you participate in group projects? *

Mark only one oval.

- ☐ Very often
- ☐ Often
- ☐ Sometimes
- ☐ Rarely
- ☐ Never

9. 9. Do you normally face problems in group communication or accountability? *

Mark only one oval.

- ☐ Yes
☐ No
☐ Sometimes

10. 10. In group projects, how often do members complete their assigned tasks on time? *

Mark only one oval.

- ☐ Always
☐ Often
☐ Sometimes
☐ Rarely
☐ Never

11. 11. How often do you feel you must remind group members about deadlines or tasks? *

Mark only one oval.

- ☐ Very often
☐ Often
☐ Sometimes
☐ Rarely
☐ Never

12. 12. How effective is your group at keeping track of each member's responsibilities?

Mark only one oval.

- ☐ Very effective
- ☐ Effective
- ☐ Neutral
- ☐ Ineffective
- ☐ Very ineffective

Section C: Perception Toward a AI-Powered Personalised Chatbox for Role-Based Guidance and Accountability In Educational Website

13. 13. A AI-Powered Personalised chatbox could help improve communication during group projects. *

Mark only one oval.

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

14. 14. An AI chatbox that gives different guidance based on roles (Leader/Member) would be beneficial. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly agree

15. 15. Having an AI chatbox remind members about tasks and deadlines would be useful. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly agree

16. 16. An AI chatbox could help reduce situations where group members forget their responsibilities. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly Agree

17. 17. A system that tracks progress and prompts members could increase accountability in group projects. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly Agree

18. 18. I believe such a system can make group work more organized. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly agree

19. 19. Using this chatbox in an educational platform can help create a more accountable group work environment. *

Mark only one oval.

- ☐ Strongly disagree
☐ Disagree
☐ Neutral
☐ Agree
☐ Strongly Agree

20. 20. I think using this chatbox can make the distribution of group responsibilities more fair and transparent. *

Mark only one oval.

- ☐ Strongly disagree
- ☐ Disagree
- ☐ Neutral
- ☐ Agree
- ☐ Strongly agree

21. 21. What features would you like to see in an AI-powered chatbox designed to support group projects and accountability? *

This content is neither created nor endorsed by Google.

Google Forms